ROBOT PENGIKUT GARIS BERBASIS MIKROKONTROLER PIC16F84

TUGAS AKHIR

Merupakan Syarat Untuk Memperoleh Gelar Sarjana Strata Satu (S1)
Pada Jurusan Teknik Elektro

Oleh :

MARFIN
NIM 2006010343

PEMINATAN ELEKTRONIKA
PROGRAM STUDI TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS PAMULANG

2010

LEMBAR PERNYATAAN HASIL KARYA SENDIRI

Saya yang bertanda tangan di bawah ini:

Nama	:	Marfin
NIM	:	2006010343
Program Studi / Peminatan	:	Teknik Elektro (Elektronika)
Fakultas	:	Teknik
Universitas	:	Universitas Pamulang

Menerangkan dengan sessungguhnya bahwa Tugas Akhir yang saya tulis benar–benar merupakan hasil penelitian atau analisis sendiri, kecuali pada bagian yang telah disebutkan sumbernya sebagai bahan rujukan.

Demikian lembar pernyataan ini saya buat dengan sebenar–benarnya, dan saya bersedia dituntut sesuai dengan hukum yang berlaku apabila ada hal–hal yang menyimpang dari pernyataan yang saya buat.

Pamulang, 10 Januari 2011

Yang membuat pernyataan

Marfin

NIM 2006010343

LEMBAR PENGESAHAN

ROBOT PENGIKUT GARIS BERBASIS MIKROKONTROLER PIC16F84

Oleh :

MARFIN
NIM 2006010343

Telah periksa dan disetujui serta dianggap layak untuk diuji secara lisan melalui Sidang Tugas Akhir.

Pamulang, 10 Januari 2011

Disetujui dan disahkan oleh:

Pembimbing I	Pembimbing II
Achmad Hindasyah, Ssi, Msi	**Ir. Suhedi**
NIDN	**NIDN 0310055301**

Mengetahui,

Ketua Jurusan Program Studi Teknik Elektro

Kartika Sekarsari, ST, MT
NIDN 0310087303

LEMBAR PENGESAHAN PERBAIKAN

ROBOT PENGIKUT GARIS BERBASIS MIKROKONTROLER PIC16F84

Oleh :

MARFIN
NIM 2006010343

Menerangkan bahwa Tugas Akhir ini telah berhasil dipertahankan dalam Sidang Tugas Akhir dan telah disetujui serta telah diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Teknik Pada Program Studi Teknik Elektro Fakultas Teknik Universitas Pamulang.

Penguji I	Penguji II
Dr. Ir. Mardiyanto, Msc	**Kartika Sekarsari, ST, MT**
NIDN 0312076003	**NIDN 0310087303**

Mengetahui,

Ketua Jurusan/Program Studi Teknik Elektro

Kartika Sekarsari, ST, MT
NIDN 0310087303

ABSTRAK

Robot pengikut garis akan menggunakan kontrol berbasis mikrokontroler PIC16F84. Penggeraknya menggunakan servo motor agar torsi yang dihasilkan lebih besar dan putarannya tidak terlalu cepat. Ada 3 komponen pokok untuk membuat robot pengikut garis. Pertama adalah Photo Sensor merk Fotek, Mikrokontroler PIC16F84 dan 2 buah servo motor. Pada photo sensor Fotek harus menggunakan Transistor atau optocoupler untuk antarmukanya. Pada rangkaian ini antarmuka photo sensor menggunakan optocoupler tipe PC817 buatan Sharp. Cara kerja rangkaian sensor ini adalah sebagai berikut. Jika sensor mendapati warna putih (sensor hidup) maka robot berputar ke kiri dengan cara hanya satu servo motor saja yang aktif. Jika sensor mendapati warna hitam (sensor mati) maka robot akan berjalan lurus ke depan dengan cara servo motor aktif keduanya.

Kata-kata Kunci : Mikrokontroler PIC16F84, Motor Servo Continuous, Photo Sensor MR-30XP, Optocoupler PC817

ABSTRACT

Line follower robot will use a PIC16F84 microcontroller as its controller. The driving force which is used is a servo motor for greater torque produced and the rotation is not too fast. There are three principal components to make a line follower robot. The first is a photo sensor Fotek brand, Microcontroller PIC16F84 and two servo motors. In the photo sensor Fotek must use a transistor or an optocoupler to the user interface. In this circuit of photo sensor interface using an optocoupler type artificial Sharp PC817. The workings of this sensor circuit is as follows. If the sensor finds the white color (sensor on) then the robot rotates to the left with only one way that active servo motor. If the sensor finds the black color (sensor off) then the robot will walk straight ahead through both active servo motor.

Key Words : Microcontroller PIC16F84, Continuous Servo Motor, Photo Sensors MR-30XP, Optocoupler PC817

KATA PENGANTAR

Alhamdulillaahirobbil'aalamiin penulis panjatkan kehadirat Allah SWT atas rahmat, nikmat, karunia dan ridhaNya sehingga saya bisa menyelesaikan tugas akhir dengan baik dan lancar. Tugas akhir ini ditujukan untuk memenuhi salah satu syarat dalam menyelesaikan jenjang studi Strata Satu (S1) pada fakultas teknik program studi teknik elektro di Universitas Pamulang.

Banyak hal yang telah penulis dapatkan selama tugas akhir ini. Hal ini tentunya tak lepas dari bantuan, dukungan, bimbingan dari berbagai pihak yang terkait. Sebagai ungkapan rasa syukur saya ingin menyampaikan ucapan terima kasih kepada :

1. Allah SWT atas semua limpahan RahmatNya. Engkaulah sponsor terbaikku Ya Rabb dari awal hingga akhir hayatku
2. Alm. Kakekku yang membesarkanku dari kecil hingga dewasa dan mendidikku untuk bisa menjadi pria yang mandiri. Terima kasih atas semua kebaikan yang telah kau berikan kepadaku dengan penuh ketulusan. Semoga kau tenang disana, Aamiin.
3. Nenek dan ibuku tercinta yang selalu sabar & tidak pernah letih untuk mensupportku dari aku belum bisa apa-apa hingga bisa berbuat banyak untuk diriku dan orang lain.
4. Pemilik Universitas Pamulang yang telah mendirikan Universitas Pamulang untuk bisa menuntut ilmu dengan biaya yang terjangkau dan Rektor Universitas Pamulang yaitu Drs. H. M. Sugeng Hidayat, Msi.
5. Kepala jurusan teknik elektro, Ibu Kartika Sekarsari, ST, MT atas semua informasi dan ilmunya.
6. Bapak Achmad Hindasyah, Ssi, Msi atas bimbingan dan waktunya juga yang telah diberikan kepadaku untuk membantu tugas akhirku hingga selesai.
7. Bapak Ir. Suhedi (salah satu dosen favoritku) atas bimbingan dan waktunya yang telah diberikan kepadaku untuk membantu tugas akhirku hingga selesai.
8. Para Dosen yang tidak bisa disebutkan satu-persatu yang telah mengajarkanku dari semester pertama hingga semester akhir. Terima kasih atas ilmunya yang sangat bermanfaat.

9. Bapak Moh. Ibnu Malik, ST atas kerjasamanya dalam pengadaan alat dan jawaban dari pertanyaan-pertanyaan yang sering aku tanyakan kepada beliau. Dari buku beliaulah sumber inspirasiku untuk membuat alat dalam tugas akhirku.
10. Teman–temaanku yang selalu bisa memberikan canda dan tawa di tengah–tengah kesibukanku sebagai pelepas lelah dalam mengerjakan tugas akhirku. Dan kebaikan teman–teman untuk membantu tugas akhirku. Dimulai dari :
    a. Luki Utomo (salah satu temanku yang mempunyai banyak kesamaan denganku, teman persaingan secara sehat di bangku kuliah yang mempunyai nilai bagus, teman futsalku, teman curhat juga, teman yang mempunyai cita–cita dan semangat yang besar).
    b. Budi Susilo (salah satu temanku yang telah sukses, sudah menikah dan mempunyai istri yang salehah). Terima kasih atas modem yang dipinjamkannya kepadaku untuk bisa mencari data melalui media internet yang memudahkanku dalam menyelesaikan tugas akhirku.
    c. Teman–teman seperjuangan dari semester pertama hingga semester akhir jurusan teknik elektro baik arus lemah maupun arus kuat yaitu: Surya, Haryanto, Meidy Fajar Wahyu, Ginanjar Supriatna, Hikmat Setiaman, Ferry Kurniawan, Feri, Masrukhan, Harry Derek, Yani Anwar, Andri Sukmawan, Rina, Ali, Yusuf Effendi, Ahmad Junaedi, Irfan, Lili Solihin, Pak Agus Sugianto, Pak Darmanto, Pak Sahroni, Mba Restiana dan yang tidak bisa disebut satu-persatu. Terima kasih untuk mau merangkulku menjadi teman kalian dan bisa diajak untuk bertukar pikiran.
    d. Teman–teman kerja di PT. Binarupa Aksara, Bimbel Focus, Gramedia, Optik Melawai dan PT. Indah Jaya.
    e. Dan teman-teman lainnya baik secara langsung maupun tidak langsung yang telah berperan serta mendukung tugas akhir ini.

Akhir kata dengan segala kerendahan hati penulis memohon maaf yang sebesar-besarnya atas kekurangan yang terdapat dalam tugas akhir ini. Untuk itulah semua saran dan kritik yang sifatnya membangun sangat diharapkan demi kesempurnaan tugas akhir ini serta besar harapan semoga tugas akhir ini dapat bermanfaat bagi semua pihak.

Pamulang, 10 Januari 2011

Penulis

DAFTAR ISI

	Halaman
Lembar Pernyataan Hasil Karya Sendiri	**i**
Lembar Pengesahan	**ii**
Lembar Pengesahan Perbaikan	**iii**
Abstrak	**iv**
Kata Pengantar	**v**
Daftar Isi	**viii**
Daftar Gambar	**xi**
Daftar Tabel	**xiii**
BAB I PENDAHULUAN	
1.1 Latar Belakang	1
1.2 Tujuan dan Manfaat	2
1.3 Rumusan Masalah	2
1.4 Batasan Masalah	2
1.5 Metodologi Penelitian	3
1.6 Sistematika Penulisan	4
BAB II DASAR TEORI	
2.1 Mikrokontroler PIC16F84	5
2.1.1 Keluarga mikrokontroler PICmicro	5
2.1.2 Fitur Mikrokontroler PIC16F84	6
2.1.3 Arsitektur Mikrokontroler PIC16F84	9
2.1.3.1 ALU (Arithmetic and Logic Unit)	11
2.1.3.2 Reset	11
2.1.3.3 Osilator (Osc)	13
2.1.3.4 Timer/Counter	15
2.1.3.5 Watchdog Timer (WDT)	18
2.1.3.6 Memori pada PIC16F84	18
2.1.3.7 Stack	20

2.1.3.8 Organisasi Register File 20
2.1.3.9 Mode Pengalamatan 21
2.1.3.10 Register Kegunaan Khusus 22
1. Register STATUS 24
2. Register INTCON 25
3. Register OPTION 26
4. Register EECON1 27
2.1.3.11 Data Memori EEPROM 28
2.1.3.12 Sela (Interupsi) 29
2.1.3.13 Port Mikrokontroler PIC16F84 30
1. Port A 31
2. Port B 32
2.2 Motor Servo 33
2.2.1 Spesifikasi dari servo continuous parallax 34
2.2.2 Mengontrol Motor Servo 34
2.2.3 Cara Menggunakan Servo 35
2.2.3.1 Membuat “horn” ke tengah 36
2.3 Photo Sensor MR-30 XP 37
2.4 Optocoupler PC817 40

BAB III PERANCANGAN SISTEM
3.1 Blok Diagram Hubungan Komponen–Komponen Utama 42
3.2 Perancangan Rangkaian Elektronik Robot Pengikut Garis Berbasis Mikrokontroler PIC16F84 44
3.3 Perancangan Mekanik Alat 45
3.4 Diagram Alir Robot Pengikut Garis Secara Keseluruhan 53
3.5 Perancangan Perangkat Lunak (Software) 54
3.5.1 Pengarah Assembler 56
3.5.2 Editor Teks 58
3.5.3 MPASM 58
3.5.5 IC-Prog 61

BAB IV HASIL DAN ANALISA

4.1 Hasil dan Analisa pengukuran tegangan output photo sensor 63

4.2 Hasil dan Analisa hubungan antara photo sensor dengan motor servo 64

4.3 Program Robot Pengikut Garis 65

BAB V KESIMPULAN DAN SARAN

5.1 Kesimpulan 69

5.2 Saran 69

Daftar Pustaka 70

Lampiran :

Lampiran A Gambar Robot Pengikut Garis Berbasis Mikrokontroller PIC16F84

Lampiran B Set Instruksi PIC16F84

DAFTAR GAMBAR

		Halaman
Gambar 2.1	Susunan pena mikrokontroler PIC16F84	8
Gambar 2.2	Arsitektur Harvard	9
Gambar 2.3	Blok diagram mikrokontroler PIC16F84	10
Gambar 2.4	PIC16F84 menggunakan RC osilator	13
Gambar 2.5	Rangkaian osilator Kristal	14
Gambar 2.6	Osilator menggunakan resonator	15
Gambar 2.7	Blok diagram dari timer/counter	16
Gambar 2.8	Timer/Counter dengan masukan detak luar	17
Gambar 2.9	Peta program memori dan stack	19
Gambar 2.10	Pemuatan Pencacah Program (PC) pada situasi yang berbeda	20
Gambar 2.11	Peta register file	21
Gambar 2.12	Pengalamatan langsung / tidak langsung	22
Gambar 2.13	Blok diagram dari pena RA0 – RA3	31
Gambar 2.14	Blok diagram dari pena RA4	32
Gambar 2.15	Servo Parallax	33
Gambar 2.16	Hubungan lebar pulsa dengan posisi “horn” servo	35
Gambar 2.17	Pulsa 1.5ms	36
Gambar 2.18	Tampilan program PICDEL	36
Gambar 2.19	Hubungan PIC-SERVO	37
Gambar 2.20	Photo Sensor MR-30XP	37
Gambar 2.21	Sirkuit Output & Diagram Koneksi	39
Gambar 2.22	Dimensi Photo Sensor MR-30XP	39
Gambar 2.23	Optocoupler PC817	40
Gambar 2.24	Dimensi Garis Besar PC817	41
Gambar 3.1	Blok Diagram Robot Pengikut Garis	42
Gambar 3.2	Skema Rangkaian Elektronik Robot Pengikut Garis berbasis Mikrokontroler PIC16F84 menggunakan program Proteus	44

Gambar 3.3	DT-PROTO 18 Pin PIC	45
Gambar 3.4	Motor Servo Continuous	46
Gambar 3.5	Photo Sensor MR-30XP	47
Gambar 3.6	Optpcoupler PC817	47
Gambar 3.7	Dua buah baterai +9 VDC	48
Gambar 3.8	Empat buah baterai +1,5 VDC	48
Gambar 3.9	Dua buah roda bagian belakang	49
Gambar 3.10	Satu buah roda bagian depan	49
Gambar 3.11	Base robot yang terbuat dari acrylic	50
Gambar 3.12	DT-HiQ PIC ICSP Programmer	50
Gambar 3.12	Konfigurasi target board untuk Mikrokontroler PIC	51
Gambar 3.13	Gambar Robot Pengikut Garis secara keseluruhan	52
Gambar 3.14	Diagram Alir Robot Pengikut Garis	53
Gambar 3.15	Langkah untuk memprogram mikrokontroler PIC16F84	59
Gambar 4.1	Lintasan yang akan diikuti oleh Robot Pengikut Garis	62

DAFTAR TABEL

		Halaman
Tabel 2.1	Contoh anggota keluarga mikrokontroler PICmicro	6
Tabel 2.2	Pena-pena pada mikrokontroler PIC16F84	8
Tabel 2.3	Kondisi bit PD# dan TO# setelah reset	12
Tabel 2.4	Nilai kapasitor untuk berbagai frekuensi	14
Tabel 2.5	Daftar Register Kegunaan khusus	23
Tabel 3.1	Keterangan dari ICSP Header	52
Tabel 4.1	Hasil pengukuran tegangan output dari photo sensor	63
Tabel 4. 2	Hasil hubungan antara photo sensor dengan motor servo	64

BAB I
PENDAHULUAN

1.1 Latar Belakang Masalah

Robot Pengikut Garis merupakan salah satu bentuk robot bergerak otonom yang banyak dirancang baik untuk penelitian, industri maupun kompetisi robot. Sesuai dengan namanya, tugas yang harus dilakukan oleh suatu robot pengikut garis adalah mengikuti garis pemandu yang dibuat dengan tingkat presisi tertentu. Kemajuan ilmu pengetahuan dan teknologi telah mendorong manusia untuk berusaha mengatasi segala permasalahan yang timbul di sekitarnya.

Keterbatasannya untuk terjun langsung melakukan proses pengendalian karena jauh, banyaknya plant yang dikendalikan dan membutuhkan waktu yang lama, lambat laun telah dapat diatasi dengan ditemukannya teknologi-teknologi baru, salah satunya adalah sistem gerak pada robot pengikut garis (*line follower robot*) yang memungkinkan manusia lebih mudah mengangkut barang atau memindahkan barang ke tempat lain dalam bidang industri serta dapat juga digunakan sebagai alat transportasi otomatis.

Sistem penggerak ini menggunakan motor servo digunakan untuk menjalankan robot pengikut garis (*line follower robot*). Motor servo merupakan penggerak utama robot ini. Dengan program PICDEL, pergerakan dari motor servo dapat diatur sesuai kebutuhan. Jadi, motor servo akan bergerak sesuai dengan garis yang telah ditentukan dan akan bergerak ketika ada masukan dari photo sensor. Mikrokontroler PIC16F84 digunakan sebagai pengontrol kerja sistem. Pada arah tertentu, Mikrokontroler PIC16F84 akan membelokkan motor servo sehingga berbelok sesuai arah dan garis.

1.2 Tujuan & Manfaat

Adapun tujuan dan manfaat dari tugas akhir ini adalah:

a. Merancang dan merealisasikan robot pengikut garis berbasis Mikrokontroler PIC16F84.
b. Pembuatan sistem penggerak pada menggunakan motor servo berbasis Mikrokontroler PIC16F84 dengan photo sensor dapat dimanfaatkan untuk alat transportasi modern serta pemindah barang pada dunia industri.

1.3 Rumusan Masalah

Perumusan masalah robot pengikut garis ini akan ditekankan pada:

1. Bagaimana merancang sebuah robot yang se-efektif mungkin untuk dapat mengikuti garis hitam pada warna dasar putih.
2. Bagaimana membuat program a*ssembly* pada mikrokontroler PIC16F84 untuk mengendalikan seluruh gerakan robot.
3. Bagaimana membuat *interface* antara mikrokontroler dengan motor servo dan photo sensor.

1.4 Batasan Masalah

Berikut adalah hal-hal yang menjadi batasan masalah pembuatan Tugas Akhir ini:

1. Penggunaan motor servo 2 buah sebagai penggerak utama.
2. Mikrokontroler yang digunakan adalah PIC16F84 sebagai sistem pemrosesan kontrol untuk semua sistem.
3. Photo sensor.
4. Pemrograman mikrokontroller PIC16F84 untuk *setting* motor servo dan photo sensor.

1.5 Metodologi Penelitian

1. Pemahaman dari buku yang saya jadikan referensi.
2. Konsultasi dengan Dosen Pembimbing, penulis buku yang saya jadikan referensi dan teman yang saya anggap ahli dalam membuat robot.
3. *Browsing* Internet.
4. Studi Kepustakaan.
5. Percobaan untuk mencari karakteristik transistor cahaya untuk mendeteksi perbedaan warna garis dengan warna latar belakangnya. Percobaan dilaksanakan dengan mengubah-ubah berbagai parameter jarak sensor, lalu mengamati tegangan keluaran photo sensor.
6. Perancangan sistem robot.
7. Pembuatan perangkat keras elektronika dan mekanik Robot Pengikut Garis.
8. Perancangan perangkat lunak menggunakan bahasa assembler.
9. Menguji dan mengambil data dari hasil perancangan. Robot harus mengikuti garis hitam tersebut dan diukur berapa tegangan yang keluar dari sensor..
10. Menganalisa hasil dan membuat kesimpulan.

1.6 Sistematika Penulisan

Untuk memudahkan dalam pemahaman isi dari tugas akhir ini maka diuraikan penulisannya sebagai berikut:

Bab I Pendahuluan

Berisi tentang latar belakang, perumusan masalah yang dirancang, pembatasan masalah yang diteliti, tujuan perancangan, manfaat perancangan, metodologi penelitian dan sistematika penulisan.

Bab II Dasar Teori

Berisi tentang teori-teori yang terkait dengan mikrokontroler PIC16F84, motor servo, photo sensor serta teori-teori terkait pendukung sistem.

Bab III Perancangan dan Realisasi

Berisi tentang rancangan dan realisasi sistem yang meliputi diagram blok perancangan sistem, *flowchart* program, perancangan mekanik alat, perancangan perangkat keras elektronika, perancangan perangkat lunak.

Bab IV Pengujian Rangkaian dan Sistem Keseluruhan

Berisi mengenai hasil perancangan dari segi fungsi maupun sistem yang digunakan dan perkiraan dari kinerja alat serta hasil dari kinerja sistem.

Bab V Kesimpulan dan Saran

Berisi tentang kesimpulan yang dapat diambil dari hasil perancangan sistem dan saran sebagai wacana pengembangan.

BAB II
DASAR TEORI

2.1 Mikrokontroler PIC16F84

Mikrokontroler PIC16F84 merupakan mikrokontroler dari keluarga PIC*micro* buatan *Microchip Inc.* Ia adalah salah satu jenis mikrokontroler dari puluhan jenis yang dikeluarkan oleh perusahaan Amerika tersebut. Keluarga mikrokontroler PIC*micro* tergolong lengkap, mulai dari mikrokontroler berpena 8 sampai 40 telah dibuat dengan kelengkapan (*feature*) yang berbeda-beda. Microchip selalu mengeluarkan *Product Line Card* yang dapat membantu seseorang tentang produk-produk terbaru yang telah dihasilkan oleh Microchip. Produk mikrokontroler dari Microchip tersebut memiliki beberapa tipe ROM internal yang dipakai. Ada yang menggunakan EEPROM, OTP (*One Time Programmable*) ataupun *Flash* memori. Mikrokontroler PIC16F84 memakai jenis memori *Flash*.

2.1.1 Keluarga mikrokontroler *PICmicro*

Microchip telah membuat puluhan jenis mikrokontroler yang tergabung dalam keluarga mikrokontroler PIC*micro*. Masing-masing mikrokontroler memiliki kekhasan masing-masing yang berbeda dalam hal jenis memori yang digunakan, ukuran memori, kegunaan dan lain-lain. Ada yang menggunakan EEPROM ada pula yang menggunakan *Flash* memori bahkan ada yang menggunakan OTP (memori sekali program). Pemilihan jenis mikrokontroler sangat diperlukan berdasarkan desain yang akan dibuat. Namun disini hanya menjelaskan penggunaan salah satu jenis mikrokontroler dari keluarga PIC*micro* yaitu PIC16F84. Berikut ini diberikan beberapa contoh mikrokontroler dari keluarga PIC*micro* beserta fitur atau kelengkapannya.

Tabel 2.1 Contoh anggota keluarga mikrokontroler PIC*micro* [1]

Nama	Program Memori	Fitur (kelengkapan)
PIC12C508A	512 x 12	OTP, 8 pin, WDT, *Internal Osc*
PIC1400	4096 x 14	ADC, *Internal Osc*, I2C
PIC16C54C	512 x 12	WDT, 12 I/0 , 40 MHz *max*
PIC16F84	1024 x 14	*Flash*, 18 pin, *Timer/Counter*, WDT

Tabel diatas berisi beberapa contoh mikrokontroler dari puluhan lainnya. Selain membuat mikrokontroler, *Microchip* juga membuat Serial EEPROM, Keelog, *Linear* OP AMP dan lain-lain.

2.1.2 Fitur Mikrokontroler PIC16F84

Dalam keluarga PIC*micro*, mikrokontroler PIC16F84 tidaklah terlalu istimewa. Ia bahkan tidak memiliki ADC *internal* ataupun *interface* untuk komunikasi serial atau I2C (*I square C*). Namun banyak orang yang memakai mikrokontroler PIC16F84 untuk memulai menggunakan mikrokontroler buatan *Microchip*. Walaupun tidak terlalu istimewa, namun mikrokontroler PIC16F84 memiliki fitur-fitur tertentu yang mungkin tidak terdapat pada mikrokontroler dari vendor lain.

Diantara fitur-fitur tersebut adalah:

a. Hanya memerlukan 35 set instruksi
b. Semua instruksi berukuran 14-bit
c. Data berukuran 8-bit
d. Memori program berukuran 1024 x 14 pada *Flash* memori (1000 kali baca-tulis)
e. 68 x 8 Register kegunaan umum (SRAM)
f. 15 Register kegunaan khusus
g. Data memori berukuran 64 x 8 pada EEPROM
h. Delapan tingkat *stack* perangkat keras

i. Empat sumber sela (*interrupt*)

- Pena sela luar (INT)
- Limpahan (*overflow*) pada *Timer* (TMR0)
- Port B (bit 4 ~ bit 7) bila berubah kondisi logika-nya.
- Penulisan data pada EEPROM telah selesai

j. Memiliki 13 buah I/O (5 port A, dan 8 port B) k. Dapat langsung mendrive LED

k. Terdapat *Timer* 8-bit dengan pembagi 8-bit

l. *Watchdog Timer* (WDT) dengan *internal* osilator

m. *Fuse* untuk kode pengaman

n. Mode pengurangan daya (*SLEEP*)

o. Dapat memakai beberapa jenis Osilator

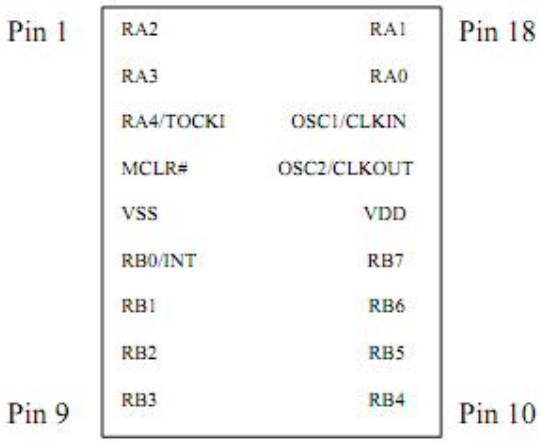
- osilator RC (RC)
- Kristal/Resonator (XT)
- Kristal kecepatan tinggi (HS)
- Kristal frekuensi rendah (LP)

p. Pemrograman didalam sistem (*In Circuit Serial Programming*).

q. Berukuran fisik hanya 18 pena.

Mikrokontroler PIC16F84 terbuat dengan teknologi CMOS (*Complementary Metal Oxide Semiconductor*) sehingga dapat didayai dengan baterai. Mikrokontroler ini dapat didayai dengan menggunakan catu daya sebesar 2V sampai 6V sehingga cukup fleksibel dalam pemakaiannya. Untuk memprogram mikrokontroler ini dapat dipakai berbagai macam programmer yang tersedia dipasar terutama adalah *Universal Programmer*. Namun *Microchip* sendiri telah membuat programmer untuk mikrokontroler ini yaitu PIC*Start*.

Susunan pena pada mikrokontroler PIC16F84 ditunjukkan pada gambar berikut ini.



Gambar 2.1 Susunan pena mikrokontroler PIC16F84 [1]

Berikut adalah penjelasan dari pena-pena pada mikrokontroler PIC16F84.

Nama Pena	Pena	Keterangan
VDD	14	Sumber daya + (positif)
VSS	5	*Ground*
OSC1/CLKIN	16	Masukan untuk detak (pin untuk kristal/resonator)
OSC2/CLKOUT	15	Output detak (pin untuk kristal/resonator)
MCLR#	4	*Master Clear* / Reset aktif rendah (Vpp untuk program)
RA4/TOCKI	3	Port A bit 4 / Masukan untuk *timer*
RA0	17	Port A bit 0
RA1	18	Port A bit 1
RA2	1	Port A bit 2
RA3	2	Port A bit 3
RB0/INT	6	Port B bit 0 / Masukan sela (*interrupt*)
RB1	7	Port B bit 1
RB2	8	Port B bit 2

RB3	9	Port B bit 3
RB4	10	Port B bit 4
RB5	11	Port B bit 5
RB6	12	Port B bit 6
RB7	13	Port B bit 7

Tabel 2.2 Pena-pena pada mikrokontroler PIC16F84 [1].

2.1.3 Arsitektur Mikrokontroler PIC16F84

Mikrokontroler PIC16F84 menggunakan arsitektur komputer Harvard. Arsitektur ini dikembangkan oleh Universitas Harvard. Pada arsitektur ini terjadi pemisahan bus antara Bus Data dengan Bus Alamat. Keuntungannya adalah dapat digunakan suatu kata panjang (*long word*) tunggal untuk perintah yang membutuhkan hanya satu lokasi pada program memori. Seperti telah diketahui mikrokontroler ini memiliki data berukuran 8-bit dan instruksi berukuran 14-bit. Instruksinya memiliki 5 langkah yaitu :

a. Pengambilan kode operasi (*fetch*)

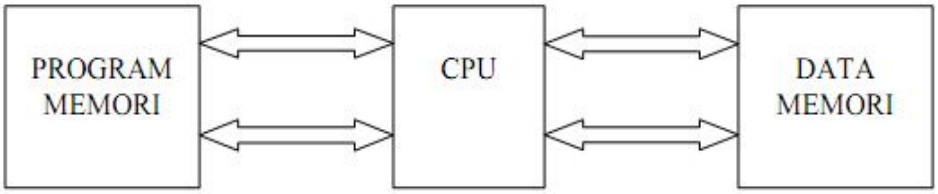
b. Penerjemahan kode operasi (*decode*)

c. Pembacaan operand (*read*)

d. Pelaksanaan kode operasi (*execute*)

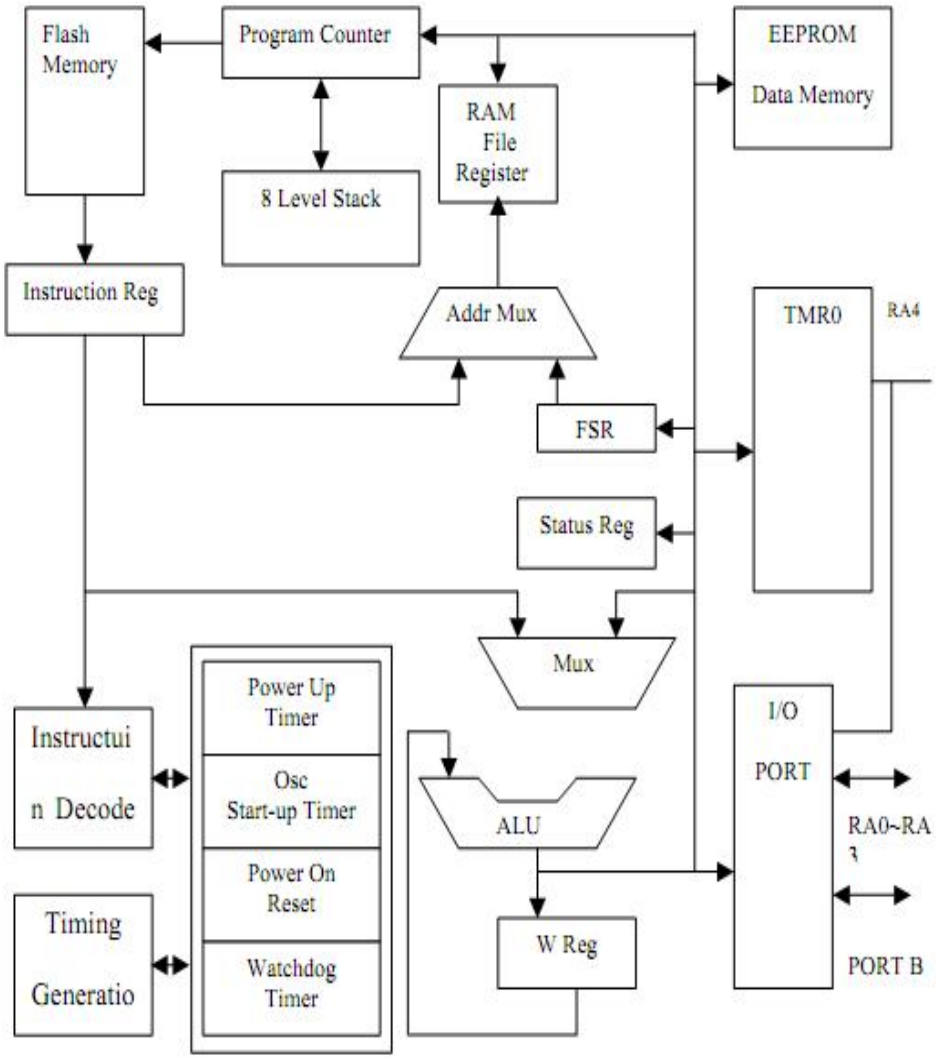
e. Penulisan operand (*write*).

Gambar berikut adalah gambar arsitektur Harvard secara umum dimana terjadi pemisahan antara bus data dan bus alamat.



Gambar 2.2 Arsitektur Harvard [1]

Mikrokontroler PIC16F84 menggunakan set instruksi yang sedikit. Hal ini karena mikrokontroler PIC16F84 termasuk dalam golongan komputer RISC (*Reduced Instruction Set Computer*). Mikrokontroler RISC melaksanakan perintah lebih cepat dari pada alat CISC namun ia memiliki kekurangan beberapa operasi akan memerlukan lebih banyak lokasi pada program memori.



Gambar 2.3 Blok diagram mikrokontroler PIC16F84 [1]

2.1.3.1 ALU (*Arithmetic and Logic Unit*)

ALU pada mikrokontroler PIC16F84 lebarnya 8-bit yang dapat melaksanakan operasi penjumlahan, pengurangan, pergeseran dan operasi logika. Kecuali disebutkan lain, semua operasi *arithmetic* memakai sistem komplemen dua. Pada operasi dua operand, biasanya satu operand adalah register kerja (*Working Register* = W). Operand lainnya adalah register *file* atau suatu konstanta. Pada instruksi dengan operand tunggal, operandnya bisa berupa register W atau suatu register *file*. W register lebarnya 8-bit tidak berada pada data memori.

2.1.3.2 *Reset*

Reset merupakan proses untuk membuat mikrokontroler kembali ke keadaan semula (*default*). Reset dapat dipandang sebagai interupsi perangkat keras yang tidak dapat ditolak oleh mikrokontroler. Pada mikrokontroler PIC16F84 dibedakan beberapa macam reset yaitu :

1) *Power On Reset* (POR)

   Pada saat pertama kali mikrokontroler dihidupkan, maka CPU akan direset untuk pertama kali. Hal ini supaya mikrokontroler berada pada keadaan awal (PC = 0000). Untuk melakukan *Reset* ini cukup dilakukan dengan menghubungkan pena MCLR# ke pena VDD (atau melalui suatu resistor 4K7 ~ 10K).

2) MCLR# *Reset* selama operasi normal.

   Pada saat mikrokontroler beroperasi dan pena MCLR# mendapat sinyal rendah, maka CPU akan segera direset.

3) MCLR# *Reset* selama *SLEEP*.

   Mikrokontroler PIC16F84 dapat menurunkan daya yang dipakainya dengan cara mengeksekusi perintah *SLEEP*. Pada saat mikrokontroler ‘tidur’ ini ia dapat dibangunkan dengan memberikan sinyal rendah pada pena MCLR#.

4) *Reset Time Out* (perhitungan waktu yang telah selesai) dari *Watchdog Timer* selama operasi normal.

5) *Reset Time Out* dari *Watchdog Timer* selama prosesor dalam mode *SLEEP*.

Beberapa register tidak direset, kondisi bit mereka tidak diketahui pada saat *Power On Reset* (POR) dan tidak berubah pada *reset* yang lainnya. Tetapi kebanyakan register direset ke kondisi resetnya pada saat terjadi POR, pada MCLR# atau WDT *reset* selama operasi normal dan pada *reset* MCLR# selama *SLEEP*. Mereka tidak dipengaruhi oleh *reset* WDT selama *SLEEP*, karena *reset* ini dianggap sebagai permulaan operasi normal.

Ada beberapa pengecualian. Pencacah Program (PC) selalu direset semua ke 0000H. Bit TO# (*Time Out*) dan bit PD# (*Power Down*) diset dan direset secara berbeda pada kondisi *reset* yang berbeda. Pada tiap kejadian *reset*, masing-masing bit TO# dan PD# memiliki nilai yang lain-lain sesuai dengan tabel berikut ini.

Tabel 2.3 Kondisi bit PD# dan TO# setelah *reset* [1]

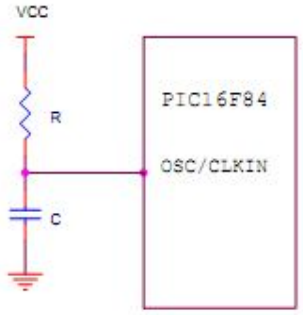
Penyebab *Reset*	**Nilai TO#**	**Nilai PD#**
WDT bangun dari *SLEEP*	0	0
WDT *time-out* (operasi normal)	0	1
MCLR# bangun dari *SLEEP*	U	0
Power Up	1	1
MCLR# *reset* selama operasi normal	U	U

Keterangan : U = tidak berubah

Bit-bit TO# dan PD# yang terdapat pada register *STATUS* dapat ditest untuk menentukan penyebab terjadinya suatu *reset*.

2.1.3.3 Osilator (Osc)

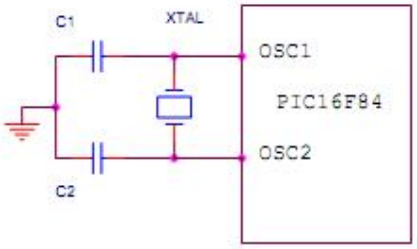
Suatu mikrokontroler tidak akan dapat bekerja jika tidak diberi detak (*clock*) melalui suatu osilator. Osilator adalah suatu rangkaian yang akan memberikan suatu pulsa pada mikrokontroler yang digunakan untuk mensinkronkan sistem yang ada dalam mikrokontroler tersebut. Untuk mengaktifkan sumber clock ini cukup dengan menambahkan rangkaian pasif saja seperti resistor/kapasitor (RC Oscilator), Kristal atau Keramik Resonator. Salah satu keistimewaan mikrokontroler PIC16F84 adalah terdapatnya 4 pilihan penggunaan osilator. Untuk pemakaian yang tidak memerlukan pewaktuan yang kritis, bahkan dapat menggunakan sepasang resitor dan kapasitor saja sebagai osilator.



Gambar 2.4 PIC16F84 menggunakan RC osilator [1]

Perlu diketahui bahwa setelah memilih suatu jenis osilator (misalnya RC Osilator), maka harus 'memberitahukan' kepada mikrokontroler pada saat proses pemrograman. Apabila osilator yang dipasang adalah RC osilator, tetapi pada saat pemrograman memilih Kristal/Resonator maka mikrokontroler tidak akan dapat bekerja. Demikian pula sebaliknya. Pada osilator RC, nilai R sebaiknya antara 3 K ohm sampai 100 K ohm. Sedangkan nilai untuk C diatas 20 pf. Pada tipe osilator ini, pena CLKOUT akan mengeluarkan detak dengan frekuensi seper-empat dari frekuensi masukannya. Untuk pemakaian mikrokontroler yang memerlukan pewaktuan kritis, maka perlu

menggunakan osilator kristal. Osilator kristal dapat dianggap yang paling baik diantara jenis osilator lainnya. Untuk memakai kristal maka ada komponen yang harus ditambahkan, yaitu 2 buah kapasitor. Rangkaian dasar untuk osilator kristal dapat dilihat pada gambar berikut ini.



Gambar 2.5 Rangkaian osilator Kristal [1]

Nilai kristal yang dapat dipakai maksimum adalah 4.0 MHz atau 10 MHz , hal ini karena mikrokontroler PIC16F84 dibuat dalam dua versi frekuensi kerja. Untuk menentukan frekuensi maksimumnya dapat dilihat pada badan mikrokontroler. Jika terdapat tulisan PIC16F84-04/P berarti frekuensi maksimum yang diijinkan adalah 4.0 MHz. Sedangkan nilai kapasitor C1 dan C2 dapat dilihat pada tabel berikut.

Tipe Osilator	Frekuensi	Nilai C1	Nilai C2
LP	32 KHz	30 pF	30 – 50pF
	100 KHz	15 pF	15 pF
	200 KHz	0 – 15 pF	0 – 15 pF
XT	100 KHz	15-30 pF	200-300pF
	200 KHz	15-30 pF	100-200 pF
	455 KHz	15-30 pF	15-100 pF
	1 MHz	15-30 pF	15-30 pF
	2 MHz	15 pF	15 pF
	4 MHz	15 pF	15 pF
HS	4 MHz	15 pF	15 pF

	10 MHz	15 pF	15 pF

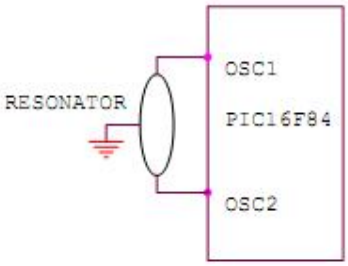
Tabel 2.4 Nilai kapasitor untuk berbagai frekuensi [1]

Untuk pemakaian yang kompak dapat menggunakan keramik resonator. Keramik resonator ini memiliki 3 terminal yang masing-masing dihubungkan dengan :

a. OSC1

b. *Ground*

c. OSC2



Gambar 2.6 Osilator menggunakan resonator [1]

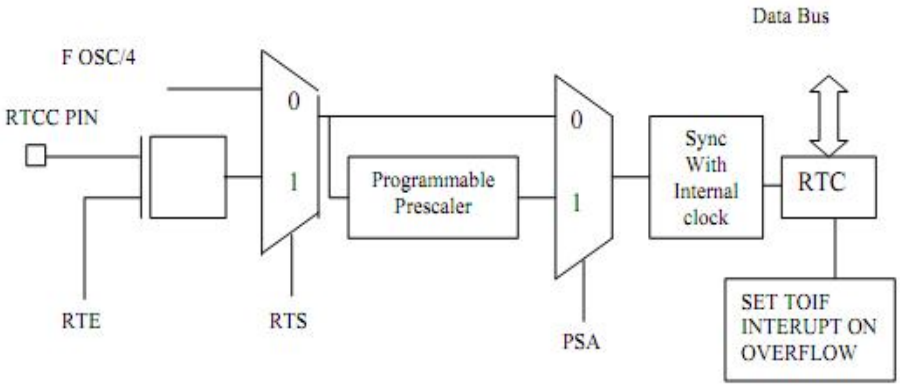
2.1.3.4 *Timer / Counter*

Mikrokontroler PIC16F84 dilengkapi dengan suatu *Timer/Counter* 8-bit. *Timer/Counter* ini dapat dipakai untuk beberapa keperluan. Diantaranya adalah untuk menghasilkan tundaan (*delay*), untuk menghitung pulsa yang masuk (fungsi *counter*), penghasil *baud rate* dan lain-lain. *Timer/Counter* ini berukuran 8-bit dan dilengkapi juga dengan *prescaler* (pembagi) 8-bit. Sifat dari *Timer/Counter* ini adalah sebagai berikut.

1) Berukuran 8-bit
2) Dapat dibaca atau ditulisi (alamat file 01H)
3) 8-bit pembagi (*prescaler*) yang dapat diprogram
4) Sumber detak dapat dipilih (detak dalam atau detak luar)

5) Terjadi sela pada limpahan dari FFH ke 00H
6) Terdapat pemilihan tebing (*Edge*) untuk detak luar

Diagram dari timer/counter yang digunakan oleh mikrokontroler PIC16F84 adalah sebagai berikut.



Gambar 2.7 Blok diagram dari *timer/counter* [1]

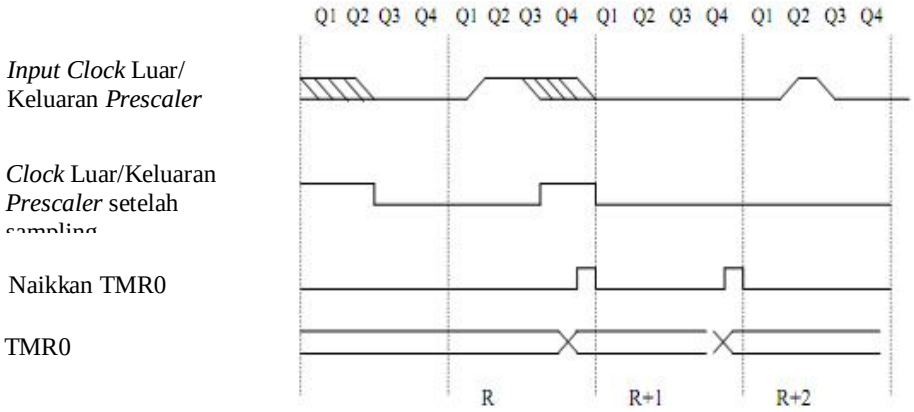
Pemilihan mode timer dilakukan dengan membersihkan (membuat '0') bit RTS pada register OPTION. Pada mode timer, modul timer akan menaikkan hitungan (*count up*) setiap siklus instruksi (bila tanpa pembagi).

Mode counter dipilih dengan mengeset (membuat '1') bit RTS (OPTION<5>). Pada mode ini counter akan menaikkan hitungan baik pada saat sisi naik atau sisi turun detak pada pena RA4/TOCKI. Pemilihan sisi naik atau sisi turun dikontrol oleh bit RTE (*OPTION* <4>). Bila bit RTE dibuat rendah maka berarti memilih sisi naik.

Pembagi pada timer ini terbagi antar modul timer dengan *Watchdog timer*. Tugas pembagi dikontrol melalui perangkat lunak dengan bit pengontrol PSA (*OPTION*<3>). Membersihkan bit PSA akan menempatkan pembagi ke timer. Pembagi tidak dapat dibaca atau

ditulisi. Saat pembagi dipakai untuk pemakaian timer, pembagian dengan nilai 1 : 2, 1 : 4 sampai 1 : 256 dapat dibuat.

Saat detak luar digunakan untuk timer, detak tersebut akan diserempakkan dengan fasa detak internal. Untuk hal tersebut masukan detak luar harus memenuhi persyaratan. Juga terdapat tundaan dari tersediannya sisi detak luar dengan naiknya timer. Berdasarkan gambar berikut, sinkronisasi dilakukan setelah proses pembagian.



Gambar 2.8 *Timer/Counter* dengan masukan detak luar [1]

Keluaran dari pembagi akan disampling dua kali setiap siklus instruksi untuk mendeteksi kenaikan (*rising*), penurunan (*falling*) sisi. Karena keluaran pembagi diserempakkan dengan detak internal, ada suatu tundaan kecil saat terjadinya sisi detak dengan terjadinya kenaikan pada modul *timer/counter*.

Pembagi (*prescaler*) merupakan suatu counter 8-bit yang digunakan sebagai pembagi bagi modul *timer/counter*. Perlu diingat bahwa hanya ada satu pembagi yang dipakai bersama antara modul *timer/counter* dengan *Watchdog Timer*. Bila pembagi telah dipakai oleh *timer*, maka *Watchdog Timer* tidak dapat menggunakannya. Bit-bit yang mengatur pembagi adalah bit PSA, bit PS2..PS0 pada register *OPTION* <3..0>.

2.1.3.5 *Watchdog Timer* (WDT)

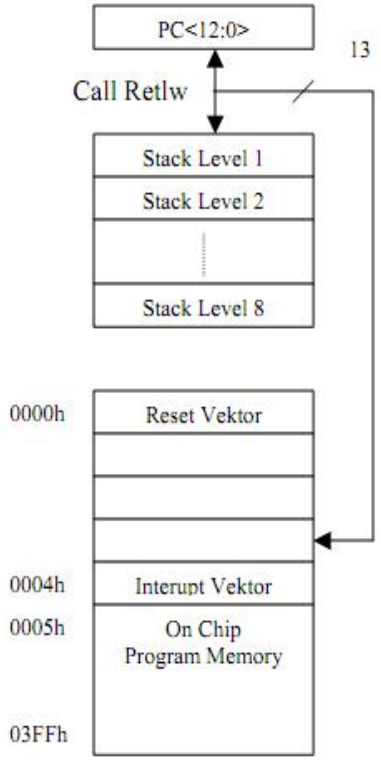
Watchdog Timer diwujudkan dalam bentuk *free running* osilator pada *chip*. Hal ini menyebabkan tidak diperlukannya komponen luar untuk mengaktifkan WDT. WDT akan tetap berjalan walaupun detak pada pena OSC1 dan OSC2 telah berhenti karena instruksi *SLEEP* dijalankan. Suatu *time-out* (perhitungan waktu yang telah selesai) dari WDT akan menghasilkan suatu kondisi *reset.* Untuk melumpuhkan WDT secara permanen dilakukan dengan membuat '0' pada *fuse* konfigurasi pada saat pemrograman mikrokontroler.

Nilai nominal periode *time-out* dari WDT adalah 18 ms (tanpa pembagi). Periode time-out dipengaruhi oleh temperatur, VDD dan variasi dari *part* ke *part*. Jika periode yang panjang diinginkan, suatu pembagi dengan pembagian sampai 1:128 dapat dilakukan dengan melalui perangkat lunak pada register *OPTION*. Periode *time-out* sampai 2,3 detik dapat direalisasikan.

2.1.3.6 Memori pada PIC16F84

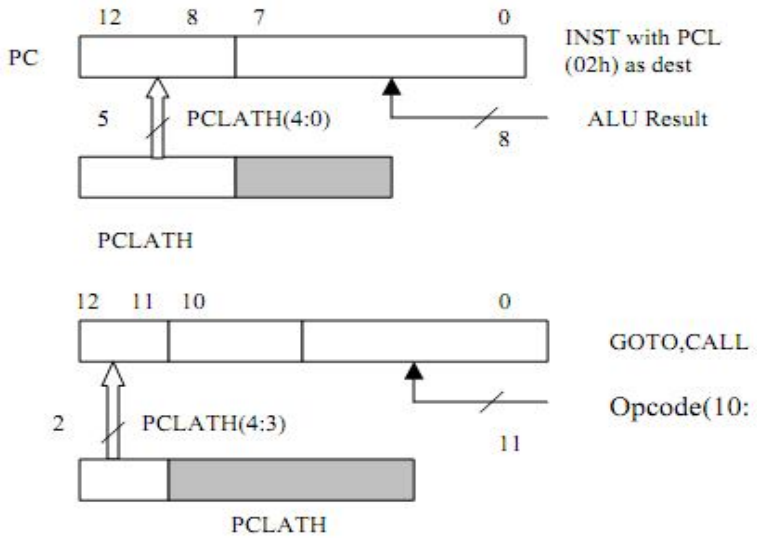
Mikrokontroler PIC16F84 memakai arsitektur Harvard yang memisahkan program memori dan data memori. Jenis *Flash* memori yang dipakai memungkinkan kita untuk memprogram-hapus sampai 1000 kali tanpa merusak alat.

Pencacah program (PC) pada mikrokontroler ini memiliki lebar 13-bit dan mampu mengalamati 8K x 14 ruang program memori. Namun hanya permulaan 1K x 14 (0000H – 03FFH) yang diwujudkan secara fisik. Vektor *reset* memiliki alamat 0000H, yang berarti bila terjadi *reset* maka pencacah program akan diisi dengan 0000H. Sedangkan vector sela adalah 0004H yang berarti bila terjadi sela, maka pencacah program akan diisi dengan 0004H. Peta program memori dapat dilihat pada gambar berikut.



Gambar 2.9 Peta program memori dan *stack* [1]

Pencacah program (PC) yang lebarnya 13-bit terdiri dari byte rendah yang diwujudkan dalam register PCL yang bersifat dapat dibaca dan ditulisi. Sedangkan byte tinggi, PCH tidak dapat langsung dibaca atau ditulisi. Byte tinggi dari PC hanya dapat ditulisi melalui register PCLATH (alamat 0AH). Saat pencacah program diisi dengan nilai baru selama pelaksanaan instruksi *CALL*, *GOTO* atau penulisan ke PCL, bit tinggi dari PC dimuati dari PCLATH.



Gambar 2.10 Pemuatan Pencacah Program (PC)
pada situasi yang berbeda [1]

2.1.3.7 *Stack*

Mikrokontroler PIC16F84 memiliki 8 ruang x 13-bit *stack* perangkat keras. Ruang stack bukan merupakan bagian ruang program atau data memori. *Stack pointer*nya tidak dapat dibaca atau ditulisi. Pencacah Program didorong ke stack saat instruksi *CALL* dilaksanakan atau suatu permintaan sela diketahui. Isi *stack* akan disembulkan (*popped*) pada pelaksanaan instruksi *RETURN*, RETLW atau RETFIE. Register PCLATH (0AH) tidak dipengaruhi oleh operasi *PUSH* dan POP.

2.1.3.8 Organisasi Register File

Register file diorganisasikan sebagai 128 x 8. Register ini dapat diakses baik secara langsung maupun tidak langsung melalui FSR (*File Select Register*). Ada beberapa bit pemilih halaman (*page*) register file pada register STATUS yang memungkinkan memilih sampai 4 halaman. Data memori hanya berukuran 2FH. Dua belas lokasi pertama digunakan untuk memetakan register fungsi khusus. Lokasi antara 0CH sampai 2FH

merupakan register kegunaan umum yang diwujudkan sebagai RAM statis. Beberapa register fungsi khusus dipetakan di halaman 1.

00	Indirect addr	Indirect addr	80
01	RTCC	OPTION	81
02	PCL	PCL	82
03	STATUS	STATUS	83
04	FSR	FSR	84
05	PORT A	TRIS A	85
06	PORT B	TRIS B	86
07			87
08	EEDATA	EECON1	88
09	EEADR	EECON2	89
0A	PCLATH	PCLATH	8A
0B	INTCON	INTCON	8B
0C	36 General Purpose	Mapped in page 0	8C
2F			AF
	PAGE 0	PAGE 1	

Gambar 2.11 Peta register file [1]

2.1.3.9 Mode Pengalamatan

Register file dapat dialamati secara langsung maupun tidak langsung. Pada kedua mode pengalamatan itu, lebih dari 512 lokasi register dapat dialamati.

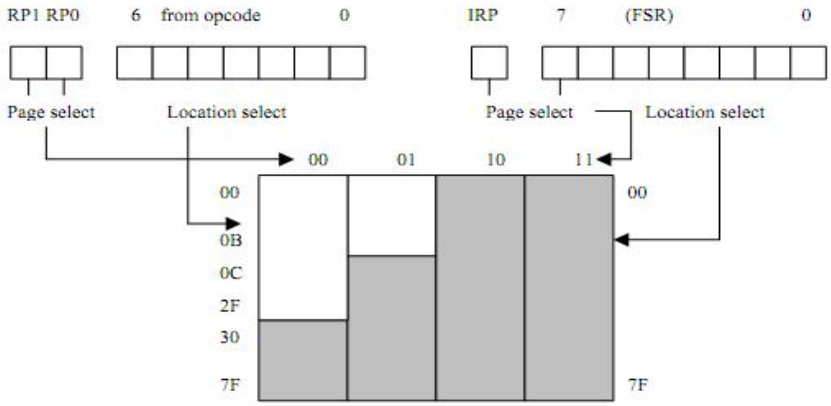
Beberapa mode pengalamatan adalah :

1) Mode pengalamatan langsung

Suatu alamat langsung 9-bit yang merupakan rangkaian 7-bit alamat langsung dari kode operasi (*op code*) dan dua bit (RP1, RP0) dari register STATUS.

2) Mode pengalamatan tak langsung

Pengalamatan tak langsung dimungkinkan dengan menggunakan alamat file 00H (INDF). Perintah yang menggunakan INDF sebagai register file sebenarnya menghubungi data yang ditunjuk oleh FSR (*File Select Register*). Membaca INDF (dirinya sendiri) secara tidak langsung akan menghasilkan 00H. Menulis ke INDF secara tidak langsung menghasilkan suatu 'no operation', walaupun bit *STATUS* akan terpengaruh. Suatu alamat 9-bit yang efektif tersusun dari rangkaian 8-bit FSR dan bit IRP pada register *STATUS*. Beberapa register fungsi khusus dipetakan pada halaman 1. Adalah sangat perlu untuk mengeset bit RP0 untuk mengalamatinya. Bit RP1 dan bit IRP pada intinya tidak diperlukan.



Gambar 2.12 Pengalamatan langsung/tidak langsung [1]

2.1.3.10 Register Kegunaan Khusus

Mikrokontroler PIC16F84 memiliki beberapa register khusus. Masing-masing register memiliki fungsi sesuai dengan namanya. Keseluruhan register dapat dilihat pada tabel berikut ini.

Tabel 2.5 Daftar Register Kegunaan khusus [1]

PAGE 0

00	IND0	*Uses contens of FSR to address data memory*
01	TMR0	8 bit *Real Time Clock Counter*
02	PCL	*Low Order 8 bits of PC*
03	*STATUS*	IRP RP1 RP0 TO PD Z DC C
04	FSR	*Indirect data memory, address pointer 0*
05	PORTA	- - - RA4/RT RA3 RA2 RA1 RA0
06	PORTB	RB7 RB6 RB5 RB4 RB3 RB2 RB1 RB0
08	EEDATA	EEPROM *Data Register*
09	EEADR	EEPROM *Address Register*
0A	PCLATH	*Holding Register for High Byte of PC*
0B	INTCON	GIE EEIE TOIE INTE RBIE TOIF INTF RBIF

PAGE 1

80	IND0	*Mapped in Page 0*
81	*OPTION*	RBPU# INTEDG RTS RTE PSA PS2 PS1 PS0
82	PCL	*Mapped in page 0*
83	*STATUS*	*Mapped in page 0*
84	FSR	*Mapped in page 0*
85	TRISA	PORTA *data direction register*
86	TRISB	PORTB *data direction register*
88	EECON1	- - - EEIF WRERR WREN WR RD
89	EECON2	*not a physical register*
8A	PCLATH	*mapped in page 0*
8B	INTCON	*mapped in page 1*

Keterangan mengenai register tersebut adalah sebagai berikut.

1. Register *STATUS*

Register ini berisi bit-bit yang merupakan status dari mikrokontroler, misalnya bit pemilihan halaman, bit *carry/borrow* dan bit *mode power down*.

Susunan bit-bit pada register STATUS

	MSB							LSB
Sifat	R/W	R/W	R/W	R	R	R/W	R/W	R/W
Bit	IRP	RP1	RP0	TO#	PD#	Z	DC	C

Keterangan :

C : Bit *Carry/Borrow#*

Untuk instruksi ADDWF, SUBWF, ADDLW dan SUBLW. Bit ini diset jika ada suatu bawaan (*carry*) dari bit msb suatu hasil operasi (*resultant*).

DC : Bit *Digit Carry/Borrow#*

Untuk instruksi ADDWF, SUBWF, ADDLW dan SUBLW. Bit ini diset jika ada suatu bawaan (*carry*) dari bit ke-4 dari suatu hasil operasi (*resultant*).

Z : Bit *Zero*

Bit ini akan diset jika hasil dari suatu operasi aritmetika atau logika adalah nol. Instruksi MOVF akan mempengaruhi bit ini.

PD# : Bit *Power Down*

Bit ini diset ke 1 selama *power up* atau oleh instruksi CLRWDT. Bit ini direset ke 0 oleh instruksi *SLEEP*.

TO# : Bit *Time-Out*

Bit ini diset ke 1 selama *power-up* dan oleh instruksi CLRWDT dan *SLEEP*. Bit ini direset ke 0 oleh *time-out* dari *Watchdog Timer*.

RP1, RP0 : Bit pemilih halaman register untuk pengalamatan langsung

00 : halaman 0 (00H – 7FH)

01 : halaman 1 (80H – FFH)

10 : halaman 2 (100H – 17FH)

11 : halaman 3 (180H – 1FFH)

Tiap halaman berisi 128 byte. Hanya RP0 yang berguna bagi PIC16F84. Bit RP1 dapat dipakai sebagai bit kegunaan umum.

IRP : Bit pemilih halaman untuk pengalamatan tak langsung

IRP : 0 - halaman 0 , 1 (00H – FFH)

1 - halaman 2 , 3 (100H – 1FFH)

Bit ini tidak dipakai untuk PIC16F84. Bit ini dapat dipakai sebagai bit kegunaan umum.

2. Register INTCON

Register ini berisi bit-bit untuk mengontrol sela (interupsi). Susunan bit-bit pada register INTCON adalah sebagai berikut.

	MSB							LSB
Sifat	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Bit	GIE	EEIE	TOIE	INTE	RBIE	TOIF	INTF	RBIF

Keterangan :

RBIF : Bendera sela untuk perubahan pada Port B. Bit ini akan diset jika masukan Port B <7:4> (Port B bit 4 sampai bit 7) berubah kondisinya. Direset oleh perangkat lunak.

INTF : Bendera sela INT. Bit ini diset saat sela INT terjadi. Reset oleh perangkat lunak.

TOIF : Bendera sela limpahan *Timer*. Bit ini diset saat terjadi limpahan hitungan pada *Timer*.

RBIE : Bit pengaktif sela RBIF. Bila RBIE = 0 akan melumpuhkan sela RBIF. Bila RBIE = 1 akan mengaktifkan sela RBIF.

INTE : Bit pengaktif sela INT. Bila INTE = 0 maka sela INTF dilumpuhkan. Bila INTE = 1 mengaktifkan sela INTF.

TOIE : Bit pengaktif sela *Timer*. Bila TOIE = 0 maka akan melumpuhkan sela TOIF. Bila TOIE = 1 akan mengaktifkan sela TOIF.

EEIE : Bit pengaktif sela penulisan EEPROM. Bila EEIE = 0 akan melumpuhkan sela EEIF. Bila EEIE = 1 akan mengaktifkan sela EEIF.

GIE : Bit pengaktif sela global. GIE = 0 sela akan dilumpuhkan. GIE = 1 sela akan diaktifkan.

3. Register OPTION

Register OPTION alamat 81H bersifat dapat dibaca dan ditulisi yang berisi berbagai bit pengontrol untuk konfigurasi pembagi (*prescaler*), pemilih sisi sela luar INT, pemilih detak *Timer* dan bit pengaktif *pull-up* pada port B.

	MSB							LSB
Sifat	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Bit	RBPU#	INTEDG	RTS	RTE	PSA	PS2	PS1	PS0

Keterangan :

PS2	PS1	PS0	Timer Rate	WDT Rate
0	0	0	1 : 2	1 : 1
0	0	1	1 : 4	1 : 2
0	1	0	1 : 8	1 : 4
0	1	1	1 : 16	1 : 8
1	0	0	1 : 32	1 : 16
1	0	1	1 : 64	1 : 32
1	1	0	1 : 128	1 : 64
1	1	1	1 : 256	1 : 128

PSA : Bit pemilih fungsi *Timer* atau *Watchdog*. PSA = 0 terpilih Timer, PSA = 1 terpilih *Watchdog Timer* .

RTE : Pemilihan tebing sinyal *Timer*

RTE = 0 : dinaikkan pada transisi rendah ke tinggi pada pena RA4/TOCKI

RTE = 1 : dinaikkan pada transisi tinggi ke rendah pada pena RA4/TOCKI

RTS : Bit pemilih sumber sinyal untuk *Timer*.

RTS = 0 : memilih detak internal

RTS = 1 : memilih transisi pada pena RA4/TOCKI (detak luar)

INTEDG : Bit pemilih tebing (*edge*) pada sela INT

INTEDG = 0 : sela terjadi pada tebing turun

INTEDG = 1 : sela terjadi pada tebing naik

RBPU# : Bit pemilih *pull-up* pada Port B

RBPU# = 0 : *pull-up* diaktifkan

RBPU# = 1 : *pull-up* dilumpuhkan.

4. Register EECON1

Register ini merupakan register pengontrol EEPROM Data Memori. Susunan bit-bit pada register ini adalah sebagai berikut.

	MSB							LSB
Sifat	U	U	U	R/W	R/W	R/W	R/W	R/W
Bit	-	-	-	EEIF	WRERR	WREN	WR	RD

Keterangan :

RD : bit pengontrol pembacaan. Jika RD = 1 memulai pembacaan EEPROM. RD dihapus oleh perangkat keras.

WR : bit pengontrol penulisan. Jika WR = 1 memulai siklus penulisan. Bit ini dihapus oleh perangkat keras jika penulisan telah selesai.

WREN : bit pengaktif penulisan EEPROM. Bila WREN = 0 melumpuhkan penulisan ke data EEPROM. Bila WREN = 1 memungkinkan penulisan ke EEPROM.

WRERR : bendera kesalahan diset jika penulisan digagalkan oleh suatu reset MCLR (selama normal atau *SLEEP*) atau oleh *reset* WDT selama operasi normal.

EEIF : bendera sela penulisan telah selesai. Set saat penulisan telah selesai. Harus direset oleh perangkat lunak.

2.1.3.11 Data Memori EEPROM

Mikrokontroler PIC16F84 dilengkapi dengan 64 x 8 data memori EEPROM yang dapat dibaca dan ditulisi selama operasi normal. Memori ini tidak secara langsung dipetakan pada ruang register file. Memori ini dapat diakses melalui 2 register yaitu EEDATA (08H) yang memegang data 8-bit untuk operasi baca/tulis dan register EEADR (09H) yang memegang alamat lokasi EEPROM yang akan diakses. Jumlah 64 byte adalah pada jangkauan 0H sampai 63H. Sebagai tambahan ada dua register kontrol yaitu EECON1 (88H) dan EECON2 (89H).

Regsiter EECON1 merupakan register kontrol dengan 5 bit bawah diimplementasikan secara fisik. 3-bit atas tidak diwujudkan dan dibaca sebagai 0. Register ini berisi bit-bit pengontrol operasi pembacaan (RD), penulisan (WR), pengaktif/pelumpuh penulisan dan lain-lain. Register EECON2 bukanlah suatu register fisik. Membaca EECON2 akan menghasilkan 0.

Untuk membaca suatu lokasi data memori harus menuliskan alamat ke register EEADR dan kemudian mengeset bit kontrol RD (EECON < 0 >). Data tersedia pada siklus berikutnya pada register EEDATA, yang dapat dibaca pada instruksi selanjutnya. Register EEDATA akan memegang nilai ini sampai penulisan berikutnya atau dituliskan ke suatu register.

Untuk menulis suatu data ke lokasi EEPROM, pertama kali harus menuliskan alamat ke register EEADR dan data ke EEDATA. Kemudian harus mengikuti urutan khusus berikut ini untuk menginisialisasi penulisan sebagai berikut.

```
Movlw 55H
Movwf EECON2
Movlw AAH
Movwf EECON2
Bsf EECON1, WR
; mulai penulisan
```

Penulisan tidak akan diinisialisasi jika urutan diatas tidak diikuti dengan waktu yang tepat. Kemudian harus melumpuhkan sela selama kode segment tersebut. Pada akhir penulisan, bit WR akan dibersihkan oleh perangkat keras dan bit EEIF diset. EEIF harus dibersihkan melalui perangkat keras.

2.1.3.12 Sela (Interupsi)

Mikrokontroler PIC16F84 memiliki empat sumber sela yaitu.

1. Sela luar pada pena RB0/INT

   Sela luar pada pena RB0/INT adalah diaktifkan oleh sisi (*edge*) baik sisi naik (jika INTEDG = 1) maupun sisi turun (INTEDG = 0). Bila suatu sisi yang sahih muncul pada pena RB0/INT, bit INTF akan diset. Sela dapat dilumpuhkan dengan membersihkan bit kontrol INTE. Bit INTF harus dibersihkan oleh program pada rutin pelayanan sela sebelum mengaktifkan lagi sela tersebut. Sela luar ini dapat membangunkan prosesor dari keadaan SLEEP jika bit INTE diset sebelum terjadinya *SLEEP*.

2. Sela limpahan pada *Timer*

   Suatu limpahan (*overflow*) yaitu dari FFH ke 00H pada *timer* akan mengeset bit TOIF. Sela ini dapat diaktifkan atau dilumpuhkan dengan mengeset/meng-*clear* bit TOIE.

3. Sela penulisan EEPROM telah selesai

   Bit bendera sela penulisan EEPROM EEIF (pada register EECON1) akan diset bila penulisan ke EEPROM telah selesai. Sela ini dapat dihalangi dengan membersihkan bit EEIE.

4. Sela karena terjadinya perubahan keadaan pada pena Port B bit 4 – bit 7. Suatu perubahan keadaan pada pena PortB <7:4> akan mengeset bit RBIF. Selain dapat diaktifkan/dilumpuhkan dengan mengeset/ membersihkan bit RBIE.

Masalah sela ini dikontrol oleh register khusus yaitu INTCON pada alamat 0BH. Register ini mencatat permintaan sela individual pada bit benderanya. Register ini juga memuat bit pengaktif sela individual atau sela global. Bit pengaktif sela global (GIE = *Global Interupt Enable*) pada register INTCON<7> mengaktifkan semua sela yang tidak dihalangi atau melumpuhkan (jika dibersihkan) semua sumber sela. Sela individual dapat dilumpuhkan melalui bit pengaktif individual pada register INTCON. Bit GIE dibersihkan pada saat terjadi *reset*. Instruksi RETFIE memungkinkan pemakai kembali dari sela dan mengaktifkan sela pada saat itu juga.

Saat suatu sela menanggapi suatu permintaan sela, bit GIE akan dibersihkan untuk mencegah sela selanjutnya yang muncul, alamat kembali akan didorong ke *stack* dan pencacah program (PC) akan diisi dengan nilai 0004H (vector sela). Untuk menentukan sumber sela dapat dilakukan dengan cara polling bit bendera sela. Bit bendera sela harus dibersihkan oleh perangkat lunak sebelum mengaktifkan sela untuk mencegah permintaan sela yang berkelanjutan.

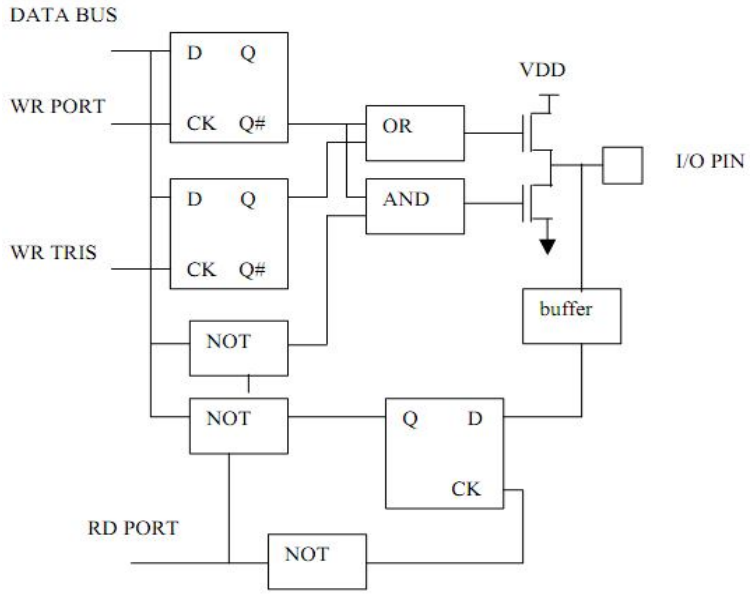
2.1.3.13 Port Mikrokontroler PIC16F84

Port pada mikrokontroler digunakan untuk berhubungan dengan dunia luar. Bila mikrokontroler hendak memantau suatu keadaan sensor atau mikrokontroler hendak menghidupkan suatu LED, maka dipakailah port. Untuk meningkatkan performa mikrokontroler dan juga untuk

menghemat jumlah pena, umumnya port pada mikrokontroler dibuat bersifat *bi-directional* (dapat bersifat sebagai masukan atau keluaran). Untuk dapat menentukan suatu port bersifat masukan atau sebagai keluaran, maka dilakukan pada tahap inisisalisasi. Inisialisasi tersebut akan memberitahukan kepada CPU mikrokontroler bahwa suatu port x telah dibuat sebagai masukan dan port lainnya sebagai keluaran. Port mikrokontroler umumnya berukuran 8 jalur (*lines*). Hal ini dikarenakan lebar data adalah 8-bit. Namun, masing-masing jalur dapat dipakai sendiri-sendiri.

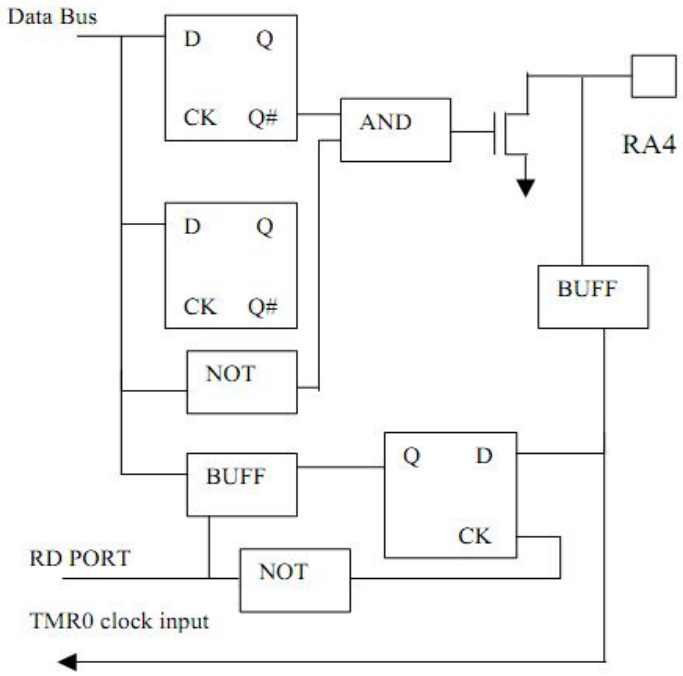
1. Port A

Port A memiliki lebar 5-bit yang dituliskan sebagai RA<4..0>. Pena RA0..RA4 bersifat *bidirectional* (dua arah) yang mana RA4 memiliki sifat keluaran kolektor terbuka. Port A merupakan register file dengan alamat 05H. Untuk mengatur Port A ini sebagai masukan atau sebagai keluaran digunakan register tertentu yaitu TRISA yang berada pada alamat 85H. Bila bit pada TRISA diisi dengan '0' maka port yang bersangkutan akan menjadi port keluaran. Sebaliknya bila diisi dengan '1' maka port tersebut menjadi port masukan.



Gambar 2.13 Blok diagram dari pena RA0 – RA3 [1]

Karena pena RA4 bersifat open kolekor, maka ia memiliki rangkaian tersendiri.



Gambar 2.14 Blok diagram dari pena RA4 [1]

2. Port B

Port B merupakan port dua arah dengan lebar 8-bit. Port B merupakan register file dengan alamat 06H. Bit pengatur arah untuk Port B dikontrol oleh register TRISB (alamat 86H). Suatu '1' pada TRISB akan mengeset pena port yang bersangkutan sebagai masukan (*input*). Membaca register Port B adalah membaca keadaan dari pena yang bersangkutan, tetapi menulis ke Port B berarti menulis ke penaut (*Port Latch*). Tiap pena Port B memiliki suatu *pull-up internal* yang dapat diaktifkan menggunakan perangkat lunak. *Pull-up* ini secara otomatis dimatikan bila port dipakai sebagai keluaran (*output*). Selanjutnya suatu bit kontrol RPBU# (register *OPTION* bit 7) dapat mematikan *pull-up* jika bit tersebut diset. *Pull-up* tersebut dilumpuhkan/dimatikan pada saat terjadi *Power On Reset*.

Port B memiliki kelengkapan sela pada perubahan masukan pada pena RB<7..4>. Saat dikonfigurasikan sebagai masukan, masukan pada pena tersebut akan disampling dan ditahan (*latch*). Masukan baru dibandingkan dengan nilai yang berada pada *latch* pada tiap siklus instruksi. Suatu keluaran aktif tinggi dihasilkan jika terjadi perbedaan antara pena dengan latch. Beberapa instruksi beroperasi sebagai "baca dilanjutkan menulis" atau "*read followed by write*". Instruksi BCF dan BSF membaca keadaan port ke CPU, melaksanakan operasi bit dan mengeluarkan hasilnya.

2.2 Motor Servo

Motor servo merupakan sebuah motor dc kecil yang diberi sistem gear dan potensiometer sehingga dia dapat menempatkan "*horn*" servo pada posisi yang dikehendaki. Motor servo ini jelas menggunakan sistim *close loop* sehingga posisi "*horn*" yang dikehendaki bisa dipertahankan. Secara umum terdapat 2 jenis motor servo. Yaitu motor servo *standard* dan motor servo *continous*. Motor servo *standard* sering dipakai pada sistim robotika misalnya untuk membuat "*Arm Robot*" (Robot Lengan) sedangkan motor servo *continuous* sering dipakai untuk Mobile Robot.

Gambar 2.15 Servo Parallax [7]

"*Horn*" pada servo ada dua jenis. Yaitu *Horn* " X" (seperti pada gambar 2.15) dan *Horn* berbentuk bulat.

2.2.1 Spesifikasi dari servo *continuous* parallax ialah:

a. Daya maksimum +6 VDC

b. Kecepatan rata-rata 60 rpm (dengan 5 +VDC dan tidak ada torsi)

c. Berat 45.0 grams/1.59oz

d. Torsi 3.40 kg-cm/47oz-in

e. Ukuran dalam mm (P x L x T) = 40.5x20.0x38.0

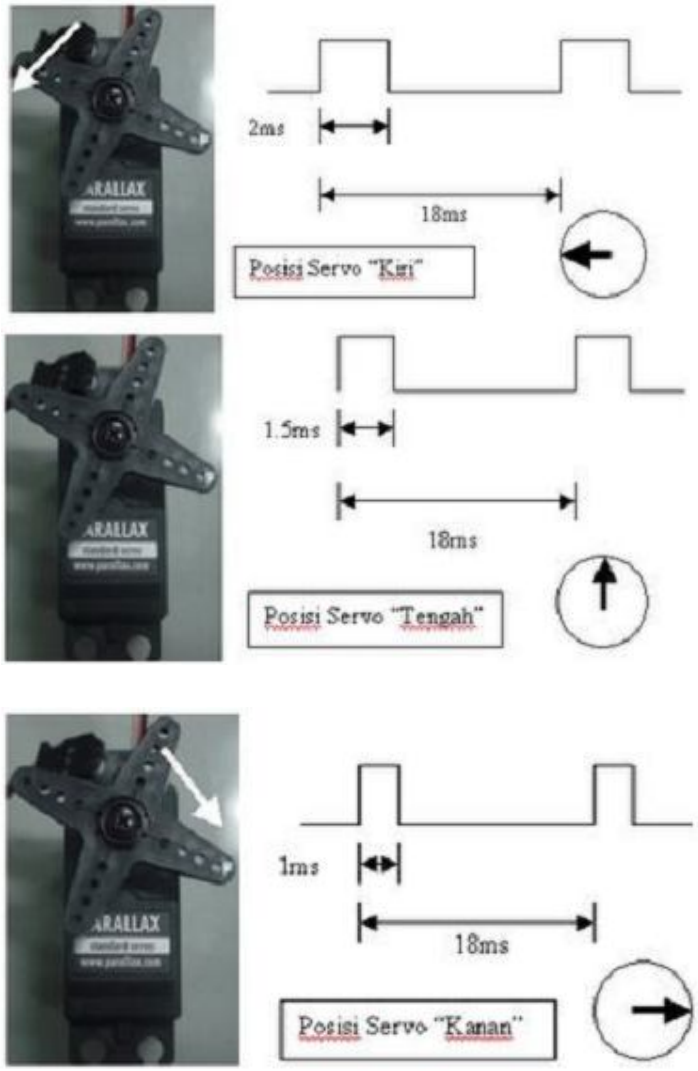
f. Ukuran dalam inci (P x L x T) = 1.60x.79x1.50

g. Penyesuaian port secara manual

2.2.2 Mengontrol Motor Servo

Penggunaan motor servo untuk bidang robotika tentu ada alasannya. Pertama adalah motor servo memiliki putaran yang lambat dan torsi yang kuat (berkat adanya sistem gear). Hal ini cocok dengan bidang robotika, bandingkan misalnya dengan motor dc biasa yang memiliki putaran cepat namun torsi rendah. Poros Motor dc yang dihubungkan langsung dengan roda, tidak akan kuat untuk menggerakkan mobile robot tersebut, demikian juga dengan motor stepper. Kedua jenis motor ini harus dihubungkan terlebih dulu dengan sistim gear agar dapat dipergunakan. Namun poros servo dapat dihubungkan langsung dengan roda.

Kedua, sistem kontrol untuk motor servo relatif sedikit (diperlukan hanya 1 jalur data saja). Hal ini tentu berbeda misalnya jika kita menggunakan motor stepper yang memerlukan jalur kontrol lebih dari 1 jalur. Oleh karena itu, tantangannya adalah bagaimana mengontrol motor servo yang hanya menggunakan 1 jalur tersebut. Karena hanya digunakan 1 jalur data untuk mengontrol motor servo, maka digunakan teknik PWM (*Pulse Width Modulation* = Modulasi Lebar Pulsa).



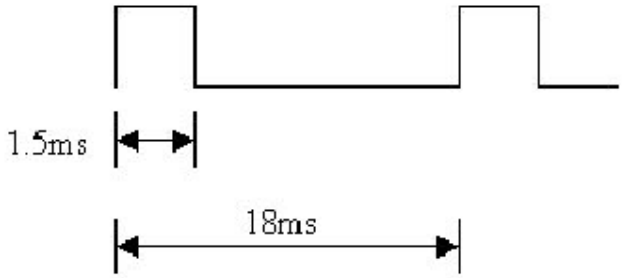
Gambar 2.16 Hubungan lebar pulsa dengan posisi "*horn*" servo [7]

2.2.3 Cara Menggunakan Servo

Untuk mencoba motor servo ini bisa menggunakan PIC16F84. Dan sangat terbantu oleh program PICDEL yang bisa menghasilkan program untuk menghasilkan tundaan (*delay*) tertentu.

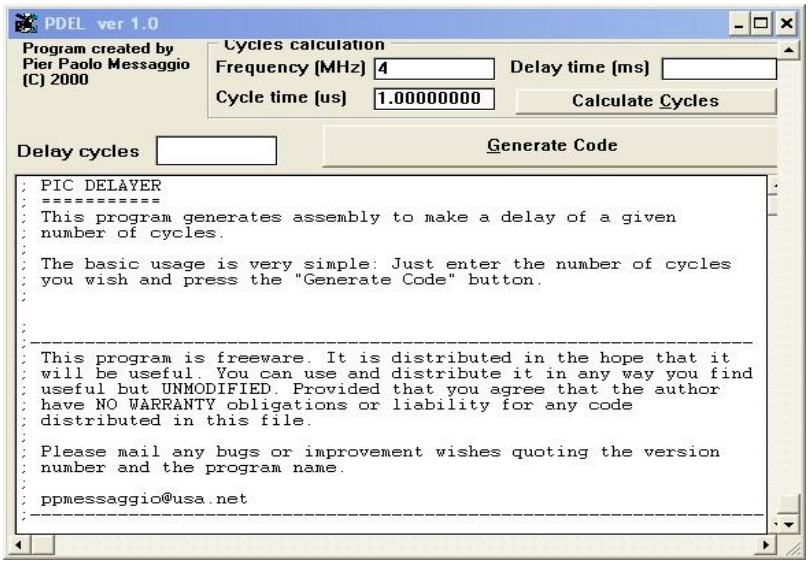
2.2.3.1 Membuat "*horn*" ke tengah

Untuk membuat horn servo motor ke tengah maka harus memberikan pulsa dengan lebar 1.5ms yang diulang-ulang. Adapun gambar pulsa tersebut adalah



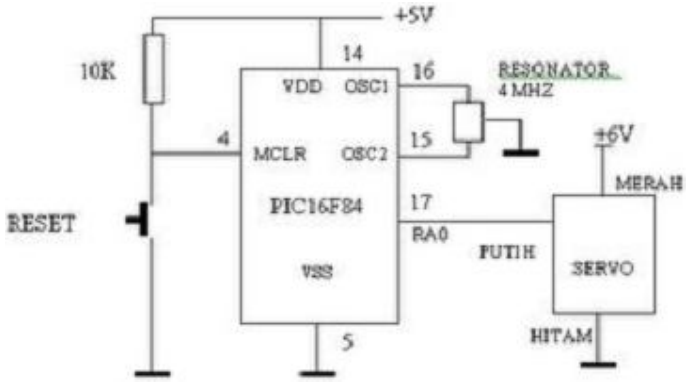
Gambar 2.17 Pulsa 1.5ms [7]

Pulsa diatas terdiri dari dua bagian, yaitu bagian yang "tinggi/high" selebar 1.5ms dan bagian yang "rendah/*low*" selebar (18ms – 1.5ms) = 16.5ms. Untuk memudahkan 16.5ms tetap akan diganti 18ms (hal ini tidak menimbulkan masalah). Jadi memerlukan tundaan waktu 1.5ms dan 18ms. Lalu buka program PICDEL dan buat tundaan 1.5ms. Caranya masukkan angka 1.5 pada kolom "*Delay Time* (ms)" setelah itu tekan tombol "*Calculate Cycle*" setelah angka pada "*Delay Cycle*" muncul baru tekan tombol "*Generated Code*".



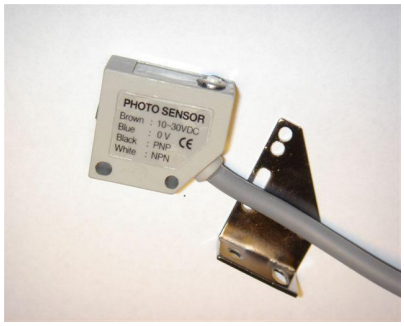
Gambar 2.18 Tampilan program PICDEL [7]

Potongan program diatas adalah potongan program untuk menghasilkan *delay* 1.5ms. Setelah itu lakukan juga agar menghasilkan potongan program untuk membuat *delay* 18ms. Untuk menghubungkan servo dengan PIC16F84 dapat dilakukan seperti gambar berikut ini.



Gambar 2.19 Hubungan PIC-SERVO [7]

2.3 Photo Sensor MR-30 XP



Gambar 2.20 Photo Sensor MR-30XP

Untuk photo sensor merk ini keluarannya tidak *open collector* seperti pada photo sensor Panasonic PANADAC 919. Jika pada Panasonic PANADAC 919 cukup diberi *pull up* 10K pada *line output*nya, maka pada photo sensor MR-30XP harus menggunakan transistor atau opto coupler untuk antar mukanya. Photo

sensor MR-30XP ini adalah tipe sensor pemantulan (*reflective type*), dipergunakan untuk mendeteksi cahaya yang dipantulkan dari objek.

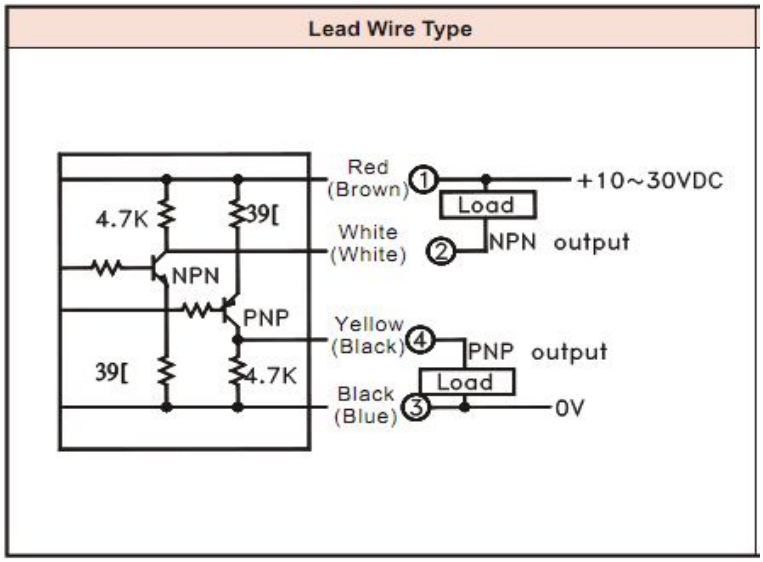
Fasilitas yang disediakan oleh photo sensor MR-30XP antara lain :

1. Deteksi non-kontak. Proses deteksi ini tidak akan mengakibatkan kerusakan baik pada target ataupun sensor.
2. Deteksi target virtual dari segala material. Proses deteksi ini didasarkan atas jumlah cahaya yang diterima, atau perubahan dalam jumlah cahaya yang dipantulkan. Metoda ini memungkinkan untuk mendeteksi target yang terbuat dari berbagai material, misalnya kaca, logam, plastik, kayu ataupun cairan.
3. Jarak deteksi yang jauh, mampu mendeteksi sampai jarak 1 (satu) meter.
4. Diskriminasi warna. Sensor ini memiliki kemampuan untuk mendeteksi cahaya dari objek berdasarkan reflektansi dan penyerapan cahaya dari warna tersebut.
5. Deteksi dengan keakuratan tinggi. Dengan sistem optikal yang unik dan sirkuit elektronik presisi memungkinkan deteksi objek dengan keakuratan tinggi.

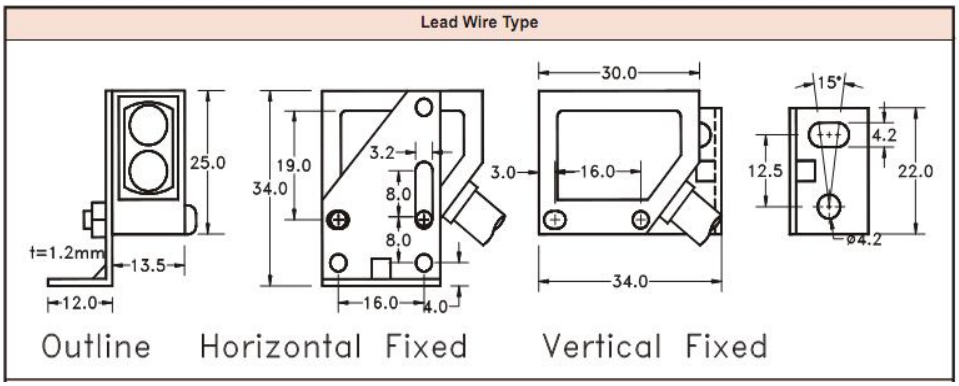
Tabel 2.6 Spesifikasi Photo Sensor MR-30XP [6]

Spesifikasi	Data
Tegangan kerja	10 ~ 30 VDC, *Ripple* <20% dari Puncak ke Puncak
Tipe Output	NPN & PNP
Status Output	NO <*Normal Open*>, Tersedia Juga Tipe NC
Arus Output	Maksimal 150mA
Arus Bocor	Maksimal 0.8mA
Tegangan Sisa	Maksimal 0.6V
Perlindungan Sirkuit	Jika terjadi korslet & polaritas terbalik
Waktu Respon	Maksimal 2ms
Pengatur Sensitivitas	Bisa diputar sampai 270^0
Emitor	LED inframerah
Penerangan	Sinar matahari <10.000 Lux. ;

	Cahaya lampu <5.000 Lux.
Histeresis	Maksimal 10% dari penginderaan jarak jauh
Bahan Kerangka	Kotak : PBT < *Polybutylene terephthalate*>, Lensa : PC
Suhu kerja	-20 C ~+60 C ; 35% ~ 85% RH
Tipe Koneksi	*Lead Wire Type* : 4 Cores / 5.0 X 2m
Keterangan	*Water Proof By Expose Filled for IP-67*

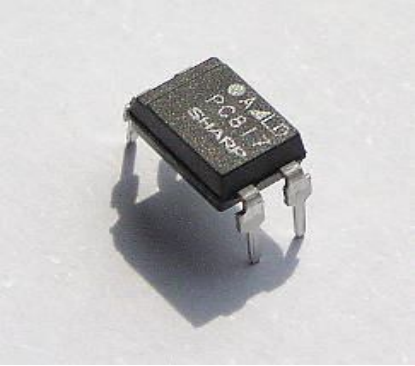


Gambar 2.21 Sirkuit output & Diagram koneksi [6]



Gambar 2.22 Dimensi Photo Sensor MR-30XP [6]

2.4 Optocoupler PC817



Gambar 2.23 Optocoupler PC817 [9]

Optocoupler adalah suatu piranti yang terdiri dari 2 bagian yaitu *transmitter* dan *receiver*, yaitu antara bagian cahaya dengan bagian deteksi sumber cahaya terpisah. Biasanya optocoupler digunakan sebagai saklar elektrik, yang bekerja secara otomatis. Optocoupler adalah suatu komponen penghubung (*coupling*) yang bekerja berdasarkan picu cahaya optic. Optocoupler terdiri dari dua bagian yaitu :

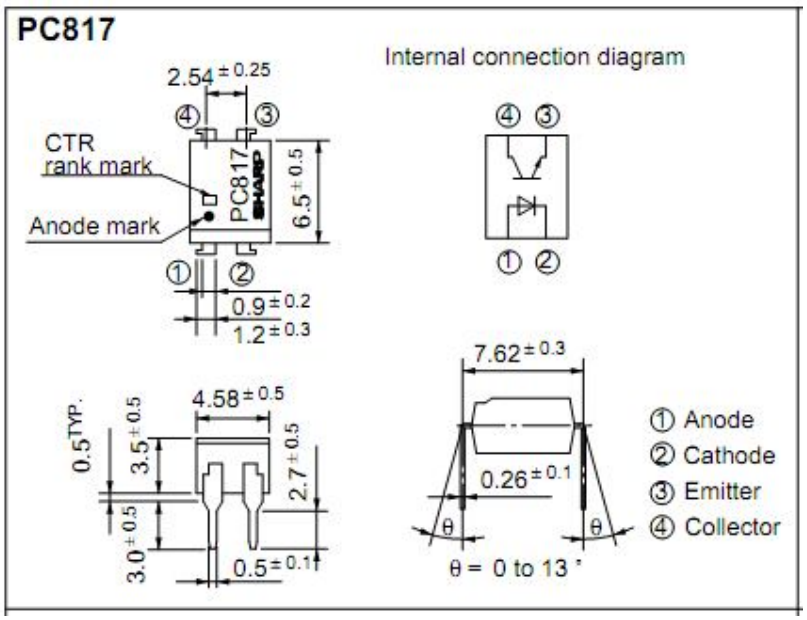
1. Pada transmitter dibangun dari sebuah LED infra merah. Jika dibandingkan dengan menggunakan LED biasa, LED infra merah memiliki ketahanan yang lebih baik terhadap sinyal tampak. Cahaya yang dipancarkan oleh LED infra merah tidak terlihat oleh mata telanjang.
2. Pada bagian *receiver* dibangun dengan dasar komponen phototransistor. Phototransistor merupakan suatu transistor yang peka terhadap tenaga cahaya. Suatu sumber cahaya menghasilkan energi panas, begitu pula dengan spektrum infra merah. Karena spekrum infra mempunyai efek panas yang lebih besar dari cahaya tampak, maka phototransistor lebih peka untuk menangkap radiasi dari sinar infra merah.

Ditinjau dari penggunaannya, fisik optocoupler dapat berbentuk bermacam-macam. Bila hanya digunakan untuk mengisolasi level tegangan atau data pada sisi *transmitter* dan sisi *receiver*, maka optocoupler ini biasanya dibuat dalam

bentuk *solid* (tidak ada ruang antara LED dan phototransistor). Sehingga sinyal listrik yang ada pada *input* dan *output* akan terisolasi. Dengan kata lain, optocoupler ini digunakan sebagai optoisolator jenis IC.

Prinsip kerja dari optocoupler adalah :

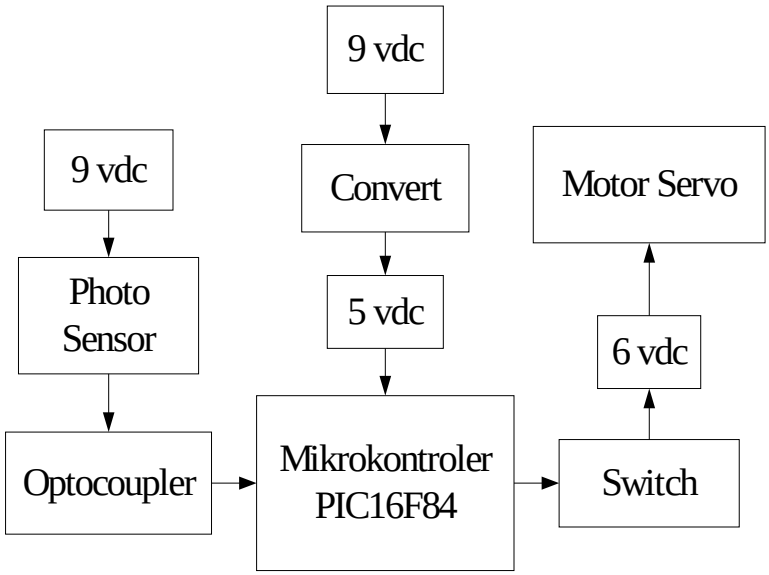
Jika antara phototransistor dan LED terhalang maka phototransistor tersebut akan *off* sehingga output dari kolektor akan berlogika *high*. Sebaliknya jika antara phototransistor dan LED tidak terhalang maka phototransistor dan LED tidak terhalang maka phototransistor tersebut akan *on* sehingga *output*-nya akan berlogika *low*.



Gambar 2.24 Dimensi garis besar PC817 [9]

BAB III
PERANCANGAN SISTEM

3.1 Blok Diagram Hubungan Komponen – Komponen Utama



Gambar 3.1 Blok Diagram Robot Pengikut Garis

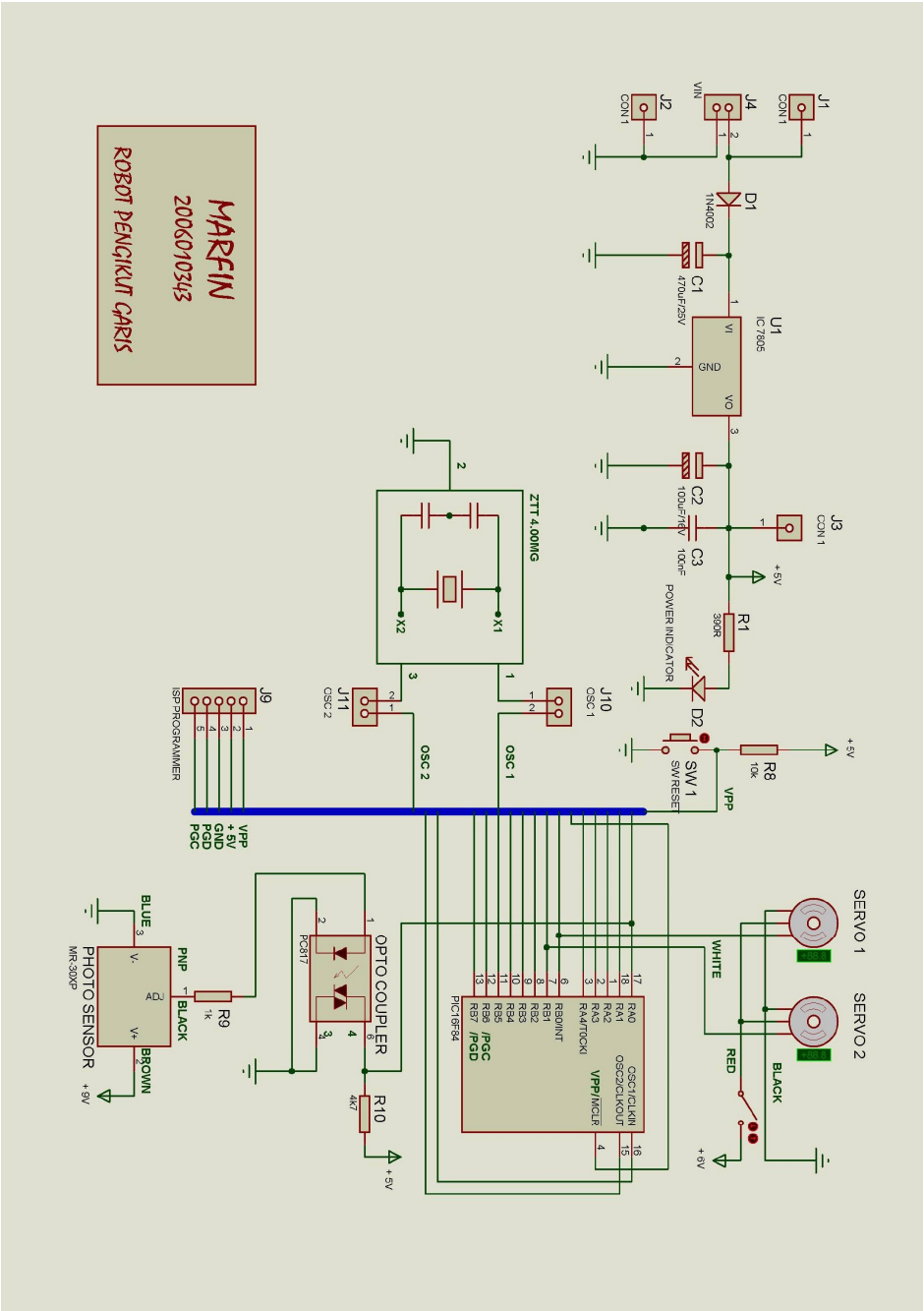
Berikut penjelasan dari blok diagram robot pengikut garis / prinsip kerjanya :

Baterai +9 VDC (baterai pertama) akan memberikan daya kepada photo sensor. Jika photo sensor mendapati dasar putih maka sinar infra merah yang dipancarkannya akan dipantulkan sehingga LED didalam optocoupler akan menyala yang menyebabkan transistor *on* dan *output* photo sensor warna kuning (PNP) akan menjadi tinggi (sekitar 7 volt untuk tegangan kerja photo sensor 9 volt). *Output* tegangan ini kemudian dihubungkan dengan masukan optocoupler PC817 melalui resistor 1K. Hal ini berakibat logika nol akan masuk ke *input* RA0.

Jika photo sensor mendapati dasar warna hitam (*line* yang harus diikuti), maka *output* PNP (kuning) akan menjadi 0 volt, sehingga LED dalam optocoupler mati dan menyebabkan logika tinggi (+5V) pada RA0 karena adanya resistor *pull up* 4K7.

Baterai +9 VDC (baterai kedua) dihubungkan ke VIN pada DT-PROTO 18 pin PIC yang memiliki tegangan *output* sebesar +5V (VCC) yang cukup untuk menyuplai daya ke mikrokontroler PIC16F84. Lalu *ON* kan *switch* untuk menghidupkan motor servo yang didayai baterai +6 VDC. Untuk menggerakkan motor servo harus menggunakan teknik PWM (*Pulse Width Modulation*) dengan bantuan program PICDEL untuk menghasilkan *delay* yang nantinya akan dimasukkan ke mikrokontroler PIC16F84. Jadi, jika tidak ada pengaturan *delay* maka pergerakan motor servo hanya sedikit sekali walaupun sudah didayai baterai sebesar +6V. Untuk pengaturan motor servo bisa dilihat di Bab 2 halaman 34.

3.2 Perancangan Rangkaian Elektronik Robot Pengikut Garis Berbasis Mikrokontroler PIC16F84



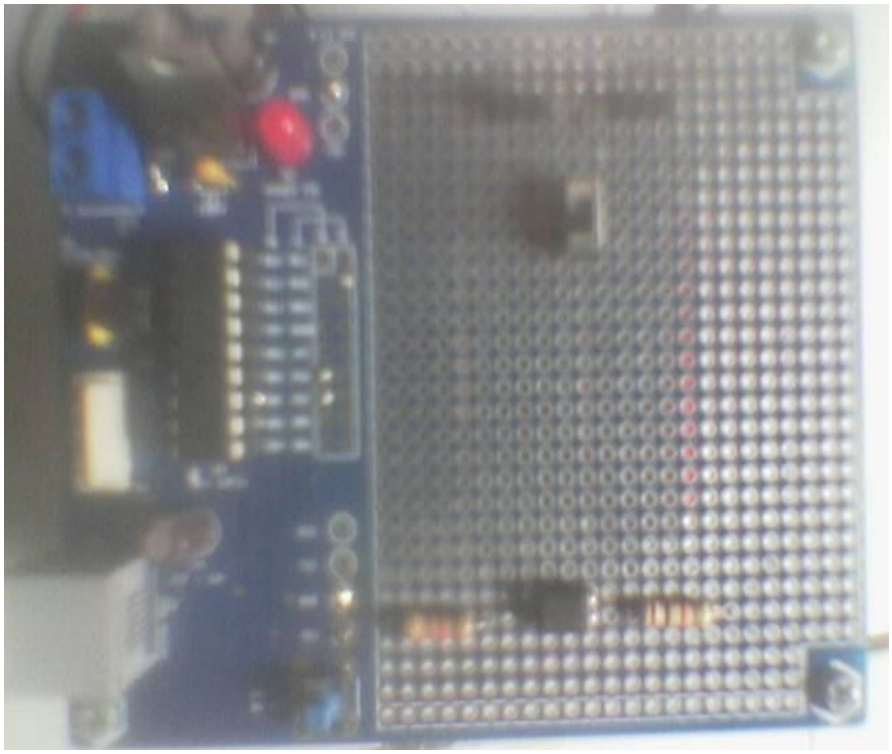
Gambar 3.2 Skema Rangkaian Elektronik Robot Pengikut Garis berbasis Mikrokontroler PIC16F84 menggunakan program Proteus

3.3 Perancangan Mekanik Alat

Berbeda dengan bab-bab sebelumnya, pada bab ini akan merancang alat yang akan dibuat. Dimana perancangan ini akan sangat berguna sebagai acuan dalam pembuatan alat dan program. Dengan perancangan ini diharapkan dapat memanfaatkan waktu sebaik-baiknya dan dapat meminimalkan kesalahan-kesalahan yang terjadi pada alat maupun programnya.

Adapun bagian – bagian untuk merancang Robot Pengikut Garis yaitu:

1. DT-PROTO 18 Pin PIC sebagai pengendali semua perangkat yang digunakan. Harga dari sistem minimum ini yaitu Rp. 120.000,00.

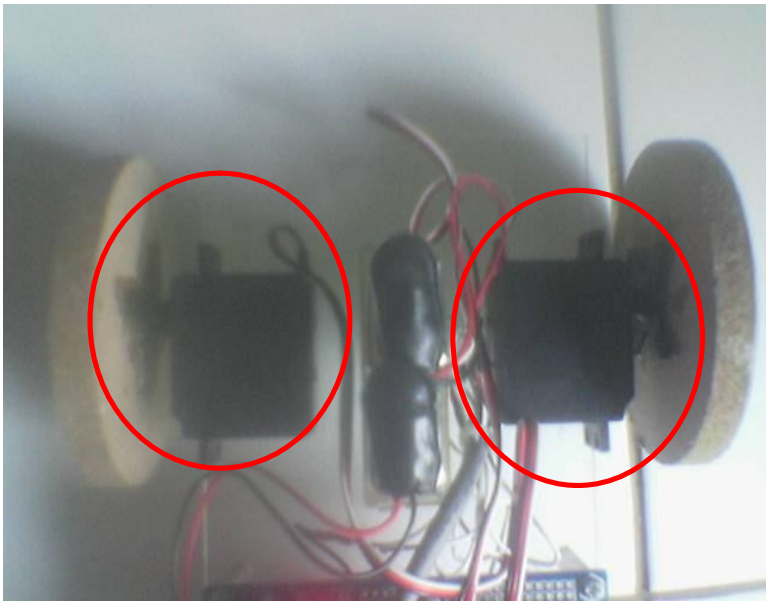


Gambar 3.3 DT-PROTO 18 Pin PIC

Spesifikasinya :

a. Mikrokontroler PIC16F84 dengan 1K byte *Program memory*, 64 byte Data EEPROM, 68 byte Data RAM, 8 bit *Timer*.
b. Mendukung varian PIC® 18 pin antara lain: PIC16C558, PIC16F627A, dan PIC16F628A.
c. Memiliki hingga 13 pin jalur *input/output* dengan kemampuan mengalirkan arus (*sink/source*) masing-masing sebesar 25 mA.
d. Tersedia sekitar 702 pad array, *through-hole*.
e. Tersedia *Ceramic Resonator* 4 MHz.
f. Tersedia Port untuk Pemrograman secara ICSP.
g. Tersedia jalur komunikasi serial UART RS-232 dengan konektor RJ11.
h. Tegangan *input* 9 - 12 VDC pada VIN dan memiliki tegangan *output* +5V (VCC).

2. Motor Servo *Continuous* buatan Parallax sebagai penggerak robot. Harga satu buah motor servo ini yaitu Rp. 190.000,00. Di dalam robot pengikut garis ini terdapat dua buah motor servo yaitu di sebelah kiri dan kanan. Untuk penjelasan lebih lengkap bisa dilihat di halaman 33.



Gambar 3.4 Motor Servo *Continuous*

Spesifikasinya :

a. Daya maksimum +6 VDC

b. Kecepatan rata-rata 60 rpm (dengan 5 +VDC dan tidak ada torsi)

c. Berat 45.0 grams/1.59oz

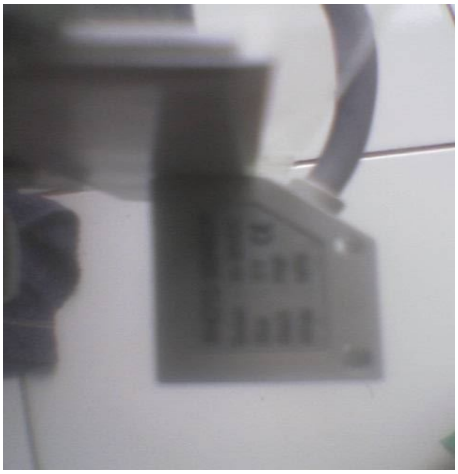
d. Torsi 3.40 kg-cm/47oz-in

e. Ukuran dalam mm (P x L x T) = 40.5x20.0x38.0

f. Ukuran dalam inci (P x L x T) = 1.60x.79x1.50

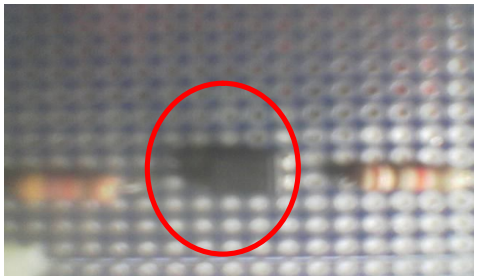
g. Penyesuaian port secara manual

3. Photo Sensor MR-30XP sebagai petunjuk jalan robot. Harga dari photo sensor ini yaitu Rp. 375.000,00. Untuk penjelasan lebih lengkap bisa dilihat di halaman 37.



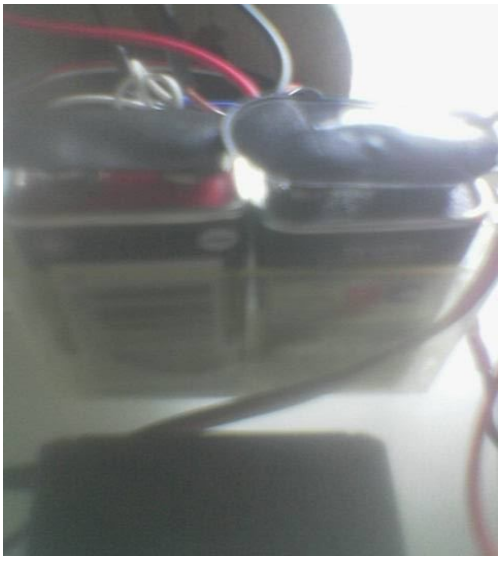
Gambar 3.5 Photo Sensor MR-30 XP

4. Optocoupler PC817 buatan Sharp sebagai *interface* dari photo sensor. Harga dari optocoupler ini cukup murah yaitu Rp.2000,00. Untuk penjelasan lebih lengkap bisa dilihat di halaman 40.



Gambar 3.6 Optocoupler PC817

5. Baterai +9 VDC dua buah untuk daya mikrokontroler dan photo sensor.



Gambar 3.7 Dua buah baterai +9 VDC

6. Baterai +1,5 VDC empat buah untuk daya motor servo.

Gambar 3.8 Empat buah baterai +1,5 VDC

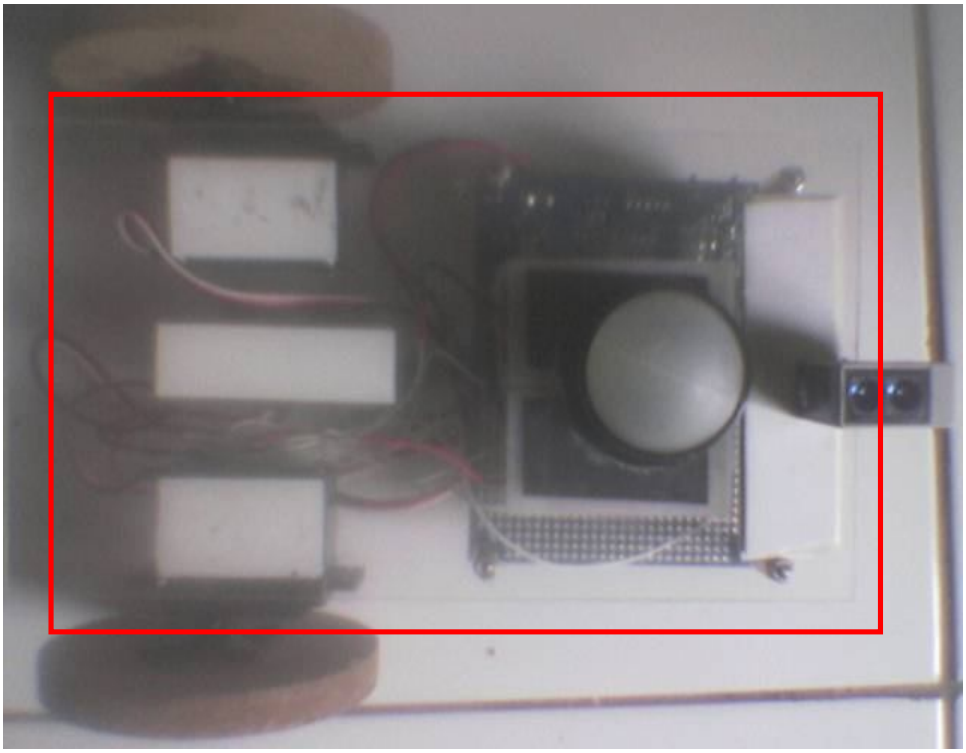
7. Dua buah roda bagian belakang pada robot pengikut garis yang terbuat dari kayu.

Gambar 3.9 Dua buah roda bagian belakang

8. Satu buah roda bagian depan pada robot pengikut garis yang diambil dari produk deodorant.

Gambar 3.10 Satu buah roda bagian depan

9. Base robot pengikut garis yang terbuat dari acrylic.



Gambar 3.11 Base robot yang terbuat dari acrylic

10. Tambahan satu lagi yaitu DT-HiQ PIC ICSP *PROGRAMMER* sebagai downloader program dari PC / Notebook yang memiliki port *parallel* (DB25) ke mikrokontroler. Harga downloader ini yaitu Rp. 110.000,00

Gambar 3.12 DT-HiQ PIC ICSP Programmer [8]

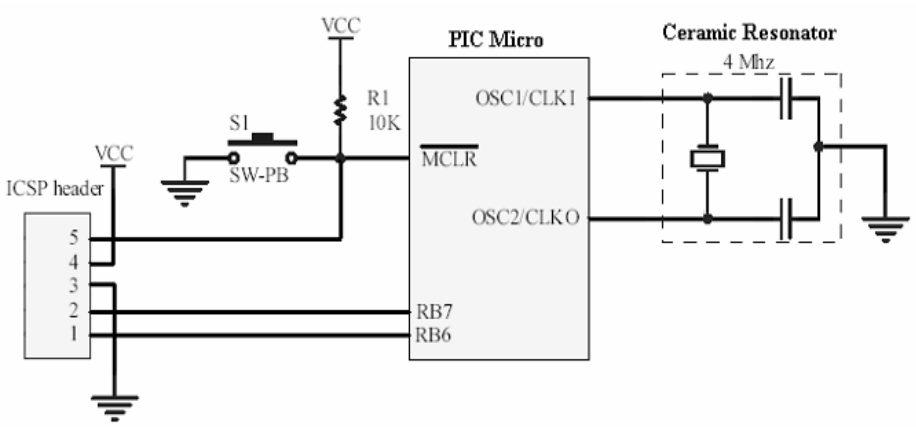
Fitur:

1. ICSP *Programmer* untuk mikrokontroler Microchip seri PIC.
2. Kompatibel dengan *Software Programmer* yang mendukung AN589 *Microchip*, seperti ProgPIC, ProgPIC2, IC-Prog.

IC yang didukung antara lain :

1. PIC12C509A	9. PIC16C66
2. PIC12F629	10. PIC16F76
3. PIC12F675	11. PIC16F873A
4. PIC16C505	12. PIC16F876A
5. PIC16F630	13. PIC16F871
6. PIC16F676	14. PIC16F874A
7. PIC16F628A	15. PIC16F877A
8. PIC16F84A	16. Dll

Konfigurasi *Target Board* untuk mikrokontroler PIC



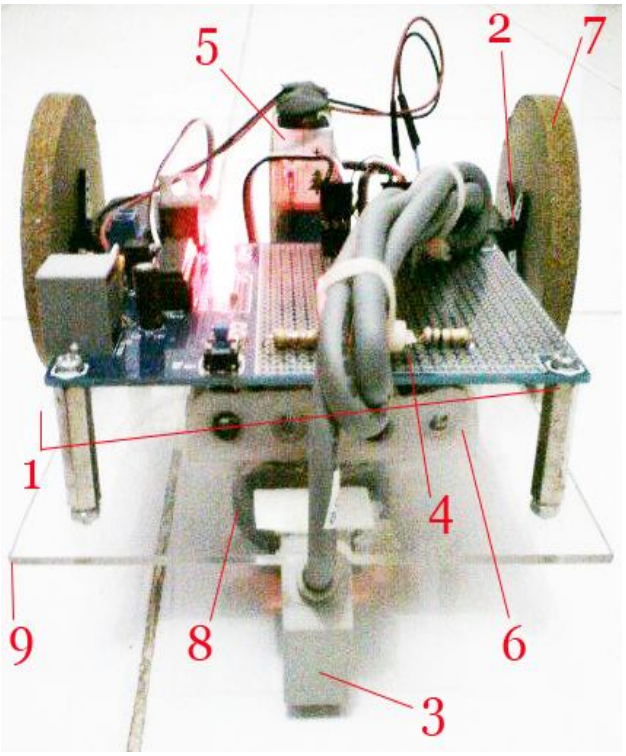
Gambar 3.12 Konfigurasi *target board* untuk Mikrokontroler PIC [8]

ICSP *header* merupakan *header* pasangan untuk konektor putih yang terdapat pada DT-HiQ PIC ICSP *Programmer*.

Keterangan dari *header* tersebut adalah sebagai berikut:

No	Fungsi
1	PGC (Clock / RB6)
2	PGD (Data / RB7)
3	GND
4	VCC (+5V)
5	VPP

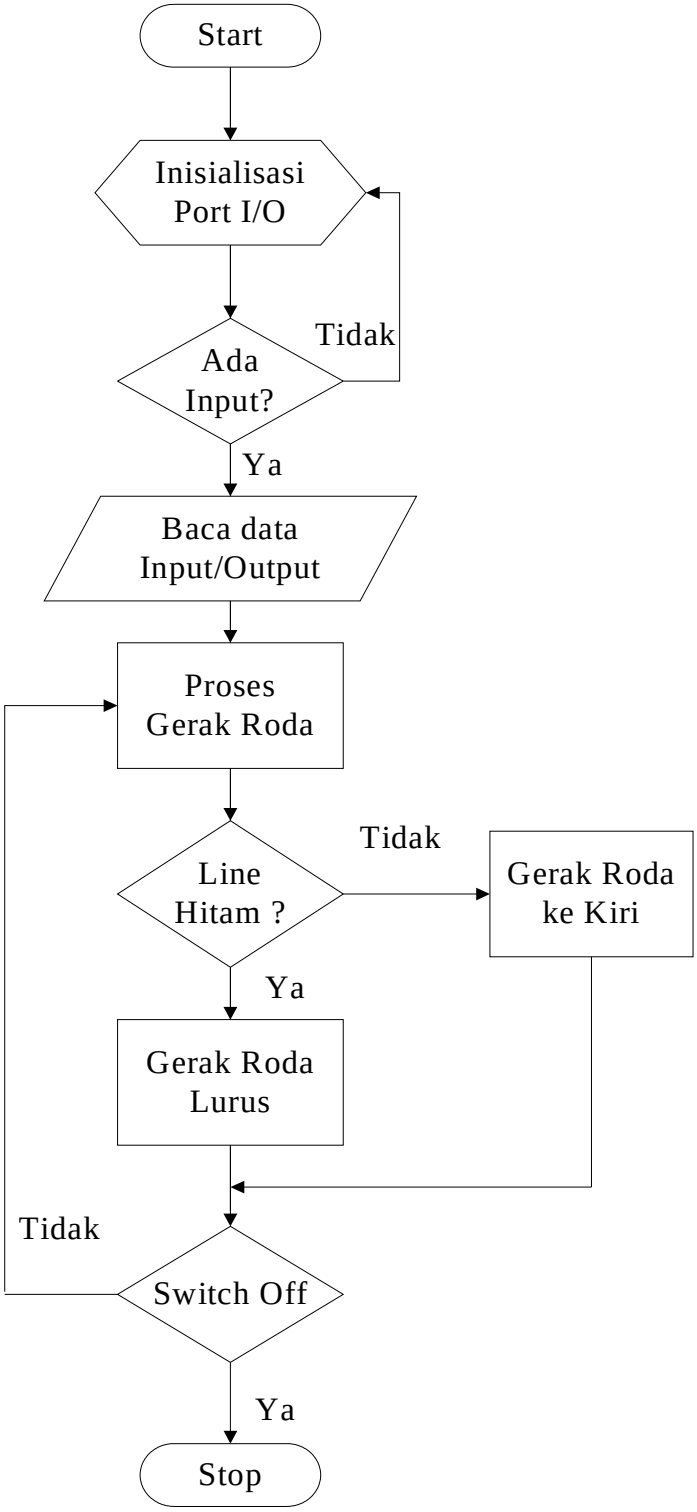
Tabel 3.1 Keterangan dari ICSP *Header* [8]



1. DT-PROTO 18 Pin PIC
2. Motor Servo Continuous
3. Photo Sensor MR-30XP
4. Optocoupler PC817
5. Baterai +9 VDC
6. Baterai +1,5 VDC
7. Roda bagian belakang
8. Roda bagian depan
9. Base robot pengikut garis

Gambar 3.13 Gambar Robot Pengikut Garis secara keseluruhan

3.4 Diagram Alir Robot Pengikut Garis Secara Keseluruhan



Gambar 3.14 Diagram Alir Robot Pengikut Garis

Keterangan dari diagram alir robot pengikut garis :

1. ***Start*** : Permulaan program
2. **Inisialisasi Port I/O** : Proses inisialisasi port – port pada mikrokontroler
3. **Ada Input** : Pernyataan dari *input* pada photo sensor. Jika Ya, maka mikrokontroler akan membaca data *input*. Jika Tidak, maka akan kembali ke inisialisasi lagi.
4. **Baca Data *Input/Output*** : Proses mikrokontroler untuk membaca data *input* pada photo sensor dan membaca data *output* pada motor servo.
5. **Proses Gerak Roda** : Proses dimana motor servo melakukan pergerakan
6. **Gerak Roda Lurus** : Proses pada motor servo untuk tetap lurus.
7. **Roda Belok ke Kiri:** Proses pada motor servo untuk belok kiri.
8. **Line Hitam** : Pernyataan pada robot pengikut garis pada dasar warna hitam. Jika Ya, maka robot pengikut garis akan lurus. Jika Tidak, maka robot pengikut garis akan berbelok ke kiri.
9. ***Switch Off*** : Pernyataan untuk mematikan motor servo. Jika Ya, motor akan berhenti. Jika Tidak, motor servo akan bergerak terus mengikuti garis.
10. ***Stop*** : Akhir dari program

3.5 Perancangan Perangkat Lunak (*Software*)

Mikrokontroler adalah alat yang bersifat *programmable*. Ia dapat diprogram untuk mengontrol suatu alat. Untuk dapat memanfaatkan mikrokontroler tersebut, ia harus diprogram sesuai dengan jenis pekerjaan yang harus ditanganinya. Proses tersebut dinamanakan memprogram.

Berbeda dengan rangkaian logika diskrit, mikrokontroler dapat dirubah. Unjuk kerjanya dengan hanya merubah program (*software*)-nya. Program suatu mikrokontroler biasanya dimasukkan kedalam suatu ROM / EPROM. Program tersebut berada dalam ROM / EPROM berwujud kode-kode biner ‘1’ dan ‘0’. Kode-kode tersebut diambil oleh mikrokontroler yang selanjutnya akan diterjemahkan dan dilaksanakan oleh mikrokontroler yang bersangkutan. Kode-kode ini harus diketahui oleh mikrokontroler tersebut. Apabila suatu program

untuk mikrokontroler X dipakai oleh mikrokontroler Y, tentu saja hal ini akan menyebabkan mikrokontroler Y tidak dapat bekerja dengan baik.

Adalah tidak mungkin bagi pembuat program untuk mengingat deretan angka ‘1’ dan ‘0’ untuk membuat suatu program. Untuk itu, untuk memudahkan membuat program maka dibuatlah suatu *mnemonic* (bantuan ingatan). *Mnemonic* merupakan kode suatu operasi yang diwujudkan dengan kata sehingga lebih mudah untuk diingat dan lebih manusiawi.

Mikrokontroler PIC16F84 merupakan mesin RISC. Dengan demikian set instruksi jumlahnya hanya sedikit saja. Tiap instruksi pada mikrokontroler PIC16F84 merupakan kata 14-bit yang terbagi menjadi suatu kode operasi yang menjelaskan tipe instruksi dan satu atau lebih operand yang nantinya menjelaskan operasi dari instruksi tersebut. Karena tiap-tiap instruksi berukuran 14-bit, maka kode operasi bisa dituliskan menjadi agak panjang. Karena itu nanti akan ditemui *mnemonic* seperti : addwf, andwf, movwf, decfsz dan lain-lain.

Set instruksi mikrokontroler PIC16F84 dibagi menjadi :

a. instruksi orientasi byte
b. instruksi orientasi bit
c. instruksi literal dan operasi kontrol

Untuk instruksi berorientasi bye, “f” menunjukkan suatu register file dan “d“ menunjukkan suatu tujuan. Bila “d“ adalah nol (0) maka suatu hasil akan ditempatkan pada register W (akumulator). Namun bila “d“ adalah satu (1) maka hasil akan ditempatkan pada suatu regiter file yang dijelaskan pada instruksi yang bersangkutan.

Untuk instruksi orientasi bit, “b“ menunjukkan medan bit yang menunjukkan nilai bit yang dipengaruhi oleh operasi tersebut. Sedangkan “f“ adalah menunjukkan nilai file dimana bit tersebut berlokasi. Untuk instruksi literal dan operasi kontrol “k“ menunjukkan suatu 8 atau 11 bit suatu nilai (konstanta) atau nilai literal.

PIC16F84 merupakan mikrokontroler yang berbasis *Flash* memori, sehingga dapat diprogram hingga 1000 kali. Hal yang menyenangkan adalah dapat memprogram Flash memori langsung pada mikrokontroler tersebut karena *Flash* memori tersebut sudah ditanamkan dalam mikrokontroler tersebut. Berbeda dengan mikrokontroler terdahulu, dimana program harus disimpan dalam EPROM terpisah.

Sebelum mendapat manfaat dari pemakaian mikrokontroler, ada beberapa langkah-langkah yang harus dipersiapkan. Pertama, adalah membuat program. Program bagi mikrokontroler PIC16F84 menggunakan bahasa tertentu yang telah diterangkan pada bab sebelumnya. Saya harus menuliskan program tersebut pada sebuah editor teks apa saja. Setelah itu proses selanjutnya adalah mengkompilasi program yang telah dibuat.

Hasil kompilasi (umumnya berektensi .hex) berisi kode-kode yang diketahui oleh mikrokontroler. *File* ini harus dimasukkan kedalam memori mikrokontroler (ROM atau *Flash* memori pada PIC16F84) melalui programmer khusus. Setelah proses pemrograman dilaksanakan, maka saatnya untuk menguji apakah mikrokontroler telah bekerja dengan benar. Bila mikrokontroler belum bekerja sebagaimana mestinya, maka beberapa hal harus diperhatikan.

a. Cek *power-supply* kepada mikrokontroler tersebut.
b. Cek osilator yang dipakai mikrokontroler tersebut.
c. Cek kode program (*program source*) pada mikrokontroler tersebut.

3.6.1 Pengarah Assembler

Sebelum membicarakan tentang pemrograman PIC16F84, akan dibahas terlebih dahulu tentang pengarah assembler (*Assembler Directive*). Pengarah ini merupakan suatu perintah assembler yang terdapat pada *file* sumber (*source file*) tetapi tidak diterjemahkan secara langsung menjadi kode operasi. Pengarah ini digunakan untuk mengontrol masukan, keluaran dan alokasi data assembler. Beberapa pengarah assembler memiliki nama dan format alternatif.

Dibawah ini akan diberikan beberapa pengarah assembler yang sering digunakan.

a. EQU

Penulisan : < *label* > equ < expr >

Keterangan : Nilai dari < expr > akan menjadi nilai bagi < *label* >

Contoh : rp0 equ 5

b. ORG

Penulisan : < *label* > org < expr >

Keterangan : mengatur awal program sebagaimana ditunjukkan oleh nilai < expr >.

Contoh : org 0x00

c. *END*

Penulisan : *end*

Keterangan : pengarah assembler ini menunjukkan akhir dari suatu program.

Contoh : *end* ; akhir program

d. *INCLUDE*

Penulisan : *include* < < *include file* > >

Include " < *include_file* >"

Keterangan : suatu *file* lain akan diikutkan dalam proses kompilasi.

Contoh : *include* "c:*include_file*\p16f84.inc"

e. *PROCESSOR*

Penulisan : *processor*

Keterangan : pengarah assembler untuk menentukan tipe prosesor yang dipakai.

Contoh : *processor* 16f84

3.6.2 *Editor* Teks

Editor teks merupakan program pengolah kata sederhana yang digunakan untuk menulis program bagi mikrokontroler. Pengolah kata yang canggih tentu tidak diperlukan, karena yang diperlukan hanya pengolah ascii teks saja. Beberapa program dibawah lingkungan Windows dapat dipakai seperti *Notepad* atau *Wordpad* (*save as text document*). Beberapa *editor* teks *under* DOS juga masih dapat dipakai, malah lebih memudahkan. *Microchip* sebagai pembuat mikrokontroler PIC16F84 menyediakan program khusus yaitu MPLAB yang selain berfungsi sebagai *editor* teks juga sebagai kompiler bagi mikrokontroler (menggunakan MPASM) dan juga sebagai *programmer* (bagi *programmer* buatan *Microchip* seperti PIC*START*, PROMATE dll).

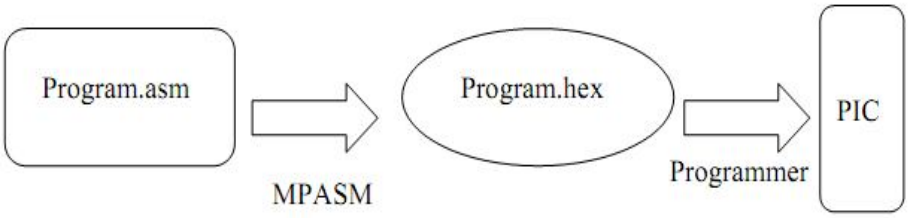
Apapun *editor* teks yang digunakan, satu hal yang harus diingat adalah berilah nama program dengan ekstensi (.asm). Hal ini memudahkan untuk diketahui *file* tersebut adalah file assembler. Pada dasarnya pemberian ektensi yang lain juga tidak disalahkan.

3.6.3 MPASM

MPASM adalah program buatan *Microchip* yang dipakai untuk mengkompilasi program yang telah dibuat bagi mikrokontroler keluarga PIC*micro*. Setelah dikompilasi, program yang telah dibuat akan diubah menjadi kode-kode tertentu (biner/hex) yang hanya diketahui oleh mikrokontroler yang bersangkutan. Karena itu file yang dihasilkan oleh MPASM akan berektensi (.hex). Berikut ini adalah salah satu contoh file hex.

```
:0A0000008316850183120514042 8FD
:00000001FF
```

Setelah mengkompilasi program yang dibuat dan telah mendapatkan *file* .hex-nya maka kini saatnya untuk memasukkan program tersebut kedalam mikrokontroler dan melihat kinerja mikrokontroler. Proses pemrograman mikrokontroler PIC16F84 dapat disederhanakan sebagai berikut.



Gambar 3.15 Langkah untuk memprogram mikrokontroler PIC16F84 [3]

Program MPASM adalah program kompilasi bukan hanya untuk mikrokontroler PIC16F84 saja, tetapi untuk keluarga mikrokontroler PIC*micro*. Program ini ada yang bekerja pada lingkungan Windows dan adapula yang bekerja pada lingkungan DOS. Untuk menjalankan program ini diperlukan komputer IBM PC/*Compatible* dengan DOS versi 5.0 keatas.

Program MPASM dapat dipakai dengan 2 cara :

a. Untuk menghasilkan kode absolut (*absolute code*) yang dapat dieksekusi secara langsung oleh mikrokontroler.
b. Untuk menghasilkan kode object (*object code*) yang dapat di-linked dengan modul terkompilasi lainnya.

File sumber yang dipakai sebagai masukan pada MPASM memiliki format tertentu. Format ini harus diikuti untuk mencegah kesalahan pada saat proses *assembly* dilakukan. Apabila terdapat kesalahan pada *file* sumber, maka pada saat dilakukan proses *assembly* akan dihasilkan pesan kesalahan Bila kesalahan terjadi, maka akan dihasilkan *file* kesalahan (*.err) yang dapat dibaca oleh program *editor* teks.

File sumber dapat dibuat dengan mengikuti aturan-aturan sebagai berikut. Tiap-tiap baris pada *file* sumber dapat berisi empat tipe informasi yaitu :

a. *Label*
b. *Mnemonic*
c. *Operand*
d. Komentar

Posisi dimana mereka ditempatkan adalah sangat penting. *Label* harus ditempatkan pada kolom pertama. *Mnemonic* dapat dimulai pada kolom kedua atau selebihnya. *Operand* mengikuti *mnemonic*. Komentar dapat diikutkan mengikuti *operand*, *mnemonic* atau *label* dan dapat dimulai pada kolom berapapun. Maksimum lebar kolom adalah 255 karakter. Satu atau lebih spasi harus dipakai untuk memisahkan *label* & *mnemonic* dan antara *mnemonic* & *operand*.

Suatu *label* harus dimulai pada kolom pertama. *Label* ini dapat diikuti oleh tanda titik dua (:), spasi, tab, atau akhir baris. *Label* harus dimulai oleh suatu huruf atau tanda garis bawah dan mungkin dapat mengandung huruf, garis bawah atau tanda tanya. *Label* dapat menggunakan karakter sampai 32 panjangnya.

Suatu *mnemonic* instruksi assembler, pemanggilan makro harus dimulai pada kolom kedua atau lebih. Jika ada label pada baris yang sama, perintah harus dipisahkan dari label tersebut oleh tanda titik dua atau oleh satu atau lebih spasi atau tab. Operand harus dipisahkan dari *mnemonic* oleh satu atau lebih spasi atau tab. Setelah tanda titik-koma oleh MPASM akan dianggap sebagai komentar dan akan diabaikan oleh MPASM.

Untuk memulai menggunakan MPASM adalah sangat mudah. Lalu tinggal memasukkan file yang akan dikompilasi melalui tombol *BROWSE*. Setelah itu tinggal tekan tombol ASSEMBLE. Apabila file sumber tidak mengandung kesalahan, maka akan ditunjukkan dengan warna hijau, tetapi jika terdapat kesalahan akan ditunjukkan dengan warna merah. Untuk mengetahui kesalahan apa yang telah kita lakukan dapat dilihat pada file *.err yang secara otomatis akan dihasilkan.

Perhatikan baik-baik pesan kesalahan pada file *.err. Kemudian perbaiki kesalahan tersebut, dan kompilasi lagi dengan MPASM. Apabila semua pesan kesalahan telah dibetulkan, maka file *.err tidak akan dihasilkan lagi.

Setelah didapat file *.hex didapat dari hasil kompilasi menggunakan MPASM, maka kini saatnya untuk memasukkan file tersebut kedalam chip mikrokontroler PIC16F84.

3.6.4 IC-Prog

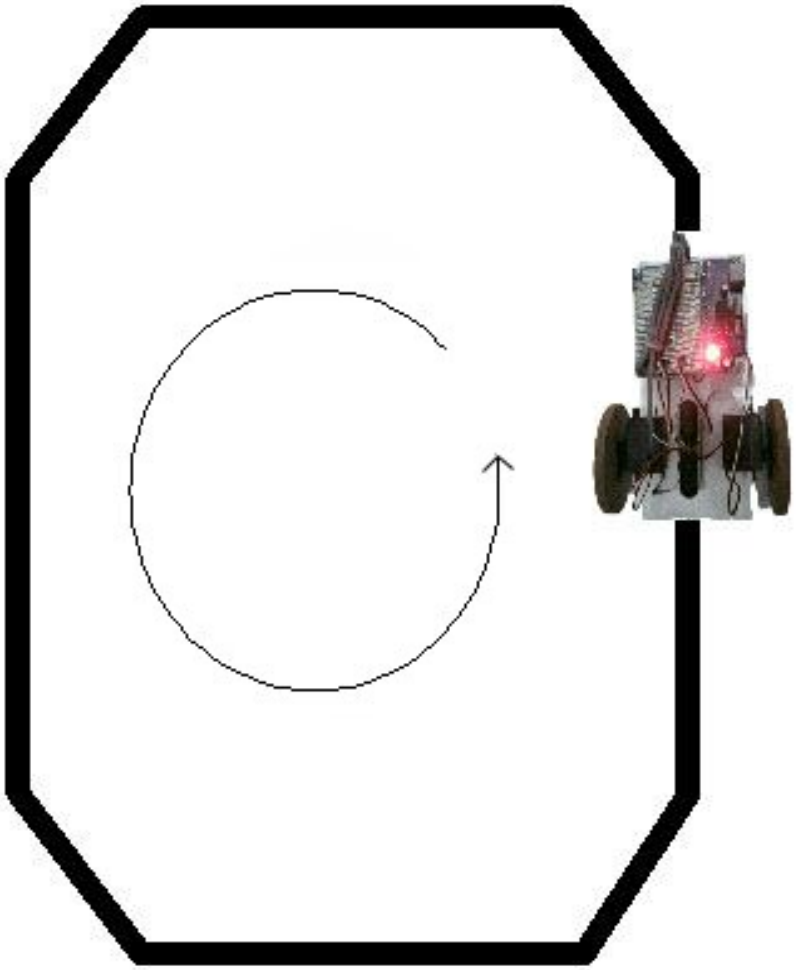
IC-Prog adalah program yang didapatkan ketika membeli ICSP *Programmer*. Fungsi dari IC-Prog ini adalah untuk mendownload program dalam format .hex yang sudah dikompilasi sebelumnya menggunakan program MPASM.

Petunjuk Penggunaan Software IC-Prog :

1. Klik/Eksekusi ICProg.exe pada CD-ROM dalam folder IC Prog (jika membeli ICSP *Programmer* maka terdapat CD-ROM yang didalamnya terdapat *software* IC-Prog atau bisa *download* di google).
2. Pada jendela *Hardware Settings* (muncul saat ICProg.exe dijalankan untuk pertama kalinya), pilih AN589 *Programmer* pada pilihan *Programmer* lalu tekan OK.
3. Untuk pengguna sistem operasi NT/2000/XP pilihlah menu *Setting-Options* dan beri tanda centang pada *Enable* NT/2000/XP *Driver* di bagian Misc.

BAB IV
HASIL DAN ANALISA

Pada bab ini merupakan kelanjutan dari bab sebelumnya. Pada Bab III banyak yang telah dijelaskan mulai dari blok diagram, *flow chart*, skema rangkaian, perancangan alat keras dan lunak. Jadi, langsung saja untuk melihat hasil dan analisa di bab ini. Di bawah ini adalah gambar lintasan robot pengikut garis. *Line* hitam adalah garis yang diikuti oleh robot pengikut garis.



Gambar 4.1 Lintasan yang akan diikuti oleh Robot Pengikut Garis

Robot pengikut garis ini akan selalu mengelilingi lintasan berwarna hitam. Kecepatan dari robot pengikut garis ini tidak bisa diubah-ubah karena program yang dimasukkan ke mikrokontroller untuk pengaturan motor servo adalah standar bagaimana motor servo itu bisa bergerak penuh (360°). Jadi, yang akan dianalisa dari robot pengikut garis ini adalah apakah robot pengikut garis ini benar-benar mengikuti lintasan warna hitam atau tidak, mengukur keluaran tegangan dari sensor saat sensor itu bertemu warna hitam dan warna putih serta hubungan antara photo sensor dengan motor servo.

4.1 Hasil dan Analisa pengukuran tegangan output photo sensor

Berikut ini adalah hasil pengukuran tegangan output photo sensor dengan menggunakan multitester digital.

Warna	Tegangan Output PNP	Tegangan Input RA0
Putih	7,7 VDC	0,12 VDC
Hitam	0,0 VDC	5,0 VDC

Tabel 4.1 Hasil pengukuran tegangan output dari photo sensor

Analisa dari hasil tabel diatas adalah saat photo sensor bertemu warna putih (lampu kuning dan lampu merah menyala pada photo sensor) saat mendapat suplai +9Vdc maka tegangan *output* dari PNP akan menjadi 7,7 Vdc dan tegangan *input* pada RA0 akan menjadi 0,12 Vdc karena tegangan *output* dari PNP akan membuat led dalam optocoupler akan menyala dan transistor menjadi *on*. Sehingga tegangan + 5Vdc dari mikrokontroller akan berlogika *low* karena arus menuju *ground* pada transistor.

Dan saat photo sensor bertemu warna hitam (lampu kuning menyala pada photo sensor) saat mendapat suplai +9Vdc maka tegangan output dari PNP akan menjadi 0 Vdc dan tegangan input pada RA0 akan menjadi 5,0 Vdc karena tidak adanya tegangan output dari PNP maka led dalam optocoupler tidak akan menyala dan transistor menjadi *cut off*. Sehingga tegangan +5Vdc dari mikrokontroller akan berlogika *high* karena arus masuk ke *input* RA0.

4.2 Hasil dan Analisa hubungan antara photo sensor dengan motor servo

Dan dibawah ini adalah hasil hubungan antara photo sensor dengan motor servo.

Warna	Motor Servo 1	Motor Servo 2
Putih (sensor *on*)	Tidak Aktif	Aktif
Hitam (sensor *off*)	Aktif	Aktif

Tabel 4. 2 Hasil hubungan antara photo sensor dengan motor servo

Analisa dari hasil hubungan antara photo sensor dengan motor servo adalah saat photo sensor bertemu warna putih maka motor servo 1 akan diam dan motor servo bergerak sehingga motor servo 1 akan menjadi tumpuan yang akan membuat motor servo 2 belok ke kiri.

Dan saat photo sensor bertemu warna hitam maka motor servo 1 dan 2 akan bergerak sehingga motor servo 1 dan 2 akan bergerak lurus.

4.3 Program Robot Pengikut Garis

Berikut ini adalah program untuk robot pengikut garis menggunakan bahasa assembler yang diketik di notepad:

```
-----------------------------------------------------------------------------------------
;Program Robot Pengikut Garis
-----------------------------------------------------------------------------------------
   processor 16f84          ;mikrokontroler yang digunakan

   status equ 0x03
   rp0    equ 0x05
   porta  equ 0x05
   trisa  equ 0x85
   portb  equ 0x06
   trisb  equ 0x06
   PDel0 equ 0x0c           ;alamat awal ram
   PDel1 equ 0x0d
   PDel2 equ 0x0e
   PDel3 equ 0x0f
   PDel4 equ 0x10
   Ulang equ 0x11
   PDel5 equ 0x12
   PDel6 equ 0x13
   PDel7 equ 0x14
   PDel8 equ 0x15

     org 0x00               ;alamat awal program
   Init
     bsf status,rp0         ;ke halaman 1
     clrf trisb             ;PortB sbg output.RB0=servo1, RB1=servo2
   movlw b'00000001'        ;RA0 sebagai input
     movwf trisa
     bcf status,rp0         ;kembali ke halaman 0
```

```
---------------------------------------------------------------------------------------
;main program
---------------------------------------------------------------------------------------
Cek_sensor
  btfss porta,0          ;cek apakah sensor
                         ;aktif (high?=hitam)
  goto Belok             ;tidak,maka belok ke kiri

Jalan
  bsf portb,0            ;aktifkan servo1 biar jalan lurus
  call Delay2ms
  bcf portb,0
  call Delay18ms

  bsf portb,1            ;aktifkan servo2 biar jalan lurus
  call Delay1ms
  bcf portb,1
  call Delay18ms
  goto Cek_sensor

Belok
  bcf portb,0            ;matikan servo1, supaya belok
  bsf portb,1            ;aktifkan servo2
  call Delay1ms
  bcf portb,1
  call Delay18ms

  goto Cek_sensor
```

```
;-----------------------------------------------------------------------
Delay1ms                        ;Delay1ms
;-----------------------------------------------------------------------
            movlw     .248      ; 1 set number of repetitions
            movwf     PDel0     ; 1 |
PLoop0      clrwdt              ; 1 clear watchdog
            decfsz    PDel0, 1  ; 1 + (1) is the time over?
            goto      PLoop0    ; 2 no, loop
PDelL1      goto PDelL2         ; 2 cycles delay
PDelL2      clrwdt              ; 1 cycle delay
            return              ; 2+2 Done
;-----------------------------------------------------------------------


;-----------------------------------------------------------------------
Delay2ms                        ;Delay 2ms
;-----------------------------------------------------------------------
            movlw     .249      ; 1 set number of repetitions
            movwf     PDel4     ; 1 |
PLoopy0     clrwdt              ; 1 clear watchdog
PDelLLLL1   goto PDelLLLL2      ; 2 cycles delay
PDelLLLL2   goto PDelLLLL3      ; 2 cycles delay
PDelLLLL3
            decfsz    PDel4, 1  ; 1 + (1) is the time over?
            goto      PLoopy0   ; 2 no, loop
PDelLLLL4   goto PDelLLLL5      ; 2 cycles delay
PDelLLLL5   clrwdt              ; 1 cycle delay
            return              ; 2+2 Done
;-----------------------------------------------------------------------
```

```
;-------------------------------------------------------------------------------
Delay18ms                       ;Delay 18ms
;-------------------------------------------------------------------------------
            movlw    .26        ; 1 set number of repetitions (B)
            movwf    PDel1      ; 1 |
PLoop1      movlw    .172       ; 1 set number of repetitions (A)
            movwf    PDel2      ; 1 |
PLoop2      clrwdt              ; 1 clear watchdog
            decfsz   PDel2, 1   ; 1 + (1) is the time over? (A)
            goto     PLoop2     ; 2 no, loop
            decfsz   PDel1, 1   ; 1 + (1) is the time over? (B)
            goto     PLoop1     ; 2 no, loop
PDelLLx1    goto PDelLLx2       ; 2 cycles delay
PDelLLx2    clrwdt              ; 1 cycle delay
            return              ; 2+2 Done
;-------------------------------------------------------------------------------


end                 ;akhir dari program
```

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Dari hasil analisa pada Bab IV disimpulkan bahwa robot pengikut garis ini dapat mengikuti garis hitam dengan baik pada saat lurus dan akan belok kiri jika bertemu warna putih. Dan pada umumnya robot pengikut garis menggunakan dua buah sensor yang bisa belok ke kiri maupun ke kanan tapi robot pengikut garis yang dibuat bisa menggunakan dengan satu buah sensor saja yang hanya bisa belok ke kiri atau ke kanan saja. Dan untuk lintasan robot pengikut garis dengan satu sensor akan lebih baik membuat lintasannya seperti loop dan tidak ada persimpangannya.

5.2 Saran

Tidak mudah untuk membuat robot pengikut garis. Diperlukan biaya yang lumayan besar dan pengetahuan yang lebih tentang robot pengikut garis ini serta mengsinkronisasikan antara hardware dengan software. Untuk itu harus banyak mencari informasi tentang membuat robot pengikut garis dan bertanya kepada orang yang ahli atau sudah pernah membuat robot pengikut garis. Karena itu salah satu point yang penting untuk membuat robot pengikut garis.

DAFTAR PUSTAKA

1. Malik, Moh. Ibnu. 2003. *Belajar Mikrokontroler PIC16F84*. Yogyakarta : Gava Media.

2. _____. 2006. *Pengantar Membuat Robot.* Yogyakarta : Gava Media.

3. _____. 2006. *Membuat Robot Dengan Mikrokontroller PIC16F84*. Yogyakarta : Gava Media.

4. Pitowarno, Endra. 2007. *Robotika : Desain, Kontrol, Dan Kecerdasan Buatan*.Yogyakarta : Andi.

5. Malik, Moh. Ibnu dan Juwana, M. Unggul. 2009. *Aneka Proyek Mikrokontroler PIC16F84/A*. Jakarta : Elex Media Komputindo.

6. **http://www.photosensor.com.tw/**

7. **http://microrobotics.blogspot.com/search/label/Mengontrol%20Motor%20Servo**

8. **http://www.innovativeelectronics.com/index_indo.php**

9. **http://www.datasheetarchive.com/optocoupler%20pc%20817-datasheet.html**